mathematics

MDPI

Article

A Zero-Touch Vulnerability Remediation Framework Based on OpenVAS, Threat Intelligence, and RAG-Enhanced Large Language Models

Cheng-Hui Hsieh, Chen-Yi Cheng and Yung-Chung Wang *

Department of Electrical Engineering, National Taipei University of Technology, Taipei 10608, Taiwan; salonpas1008@gmail.com (C.-H.H.); jessica.cheng@securityscorecard.io (C.-Y.C.)
* Correspondence: ycwang@ntut.edu.tw; Tel.: +886-928210811

Abstract

Vulnerability disclosures are outpacing manual remediation capacity. We present a Zero-Touch Vulnerability Remediation Framework combining OpenVAS scanning, multi-source threat intelligence, and Large Language Models (LLMs) enhanced through Retrieval-Augmented Generation (RAG). The Scanning Layer normalizes findings into structured JSON; the AI Decision Layer applies hybrid FAISS + BM25 retrieval, dual-LLM verification (a primary generator checked by a gpt-4o auxiliary verifier), and confidence-based routing; the Orchestration Layer executes validated patches via CI/CD pipelines with automated rollback. On 350 real-world vulnerability cases across five GPT-family models, the full Prompt + RAG pipeline raised accuracy from 52.0% to 76.7–82.6% (all $p < 0.001$, Cohen's h = 0.51–0.68) and reduced hallucination from 23.4% to 7.8%. Confidence routing routed 34.9% of cases to the high-confidence auto-execution tier, yielding a 4.1% rollback rate and zero service outages. The framework addresses the most relevant categories of the OWASP LLM Top 10 and lays groundwork for enterprise-scale, Zero-Touch vulnerability management.

Keywords: zero-touch vulnerability remediation; OpenVAS; retrieval-augmented generation (RAG); threat intelligence; large language models (LLMs); prompt engineering

MSC: 68T07; 68M25

1. Introduction

The National Vulnerability Database (NVD) now catalogs over 200,000 CVEs, and the rate of new disclosures continues to accelerate [1]. Unpatched known vulnerabilities remain a leading cause of serious security incidents according to Verizon's Data Breach Investigations Report (DBIR)—the problem is not detection but remediation delay [2]. The workforce gap makes this worse: ISC2's Cybersecurity Workforce Study puts the global talent shortage in cybersecurity at millions of unfilled positions [3]. Manual patching workflows cannot keep pace. Industry reports indicate that median enterprise patching cycles commonly span 60–90 days across many organizational settings [2], a timeline that leaves organizations exposed far longer than the threat landscape permits.

ETSI's Zero-touch Network and Service Management (ZSM) paradigm, developed for 5G/B5G systems, defines a closed-loop autonomy model based on perceive—analyze—decide—act cycles [4]. In parallel, Large Language Models (LLMs) have shown practical utility in cybersecurity tasks such as vulnerability analysis, patch suggestion, and threat

Check for updates

Academic Editor: Antanas Cenys

Received: 16 February 2026
Revised: 13 March 2026
Accepted: 18 March 2026
Published: 22 March 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

intelligence processing [5–8]. RAG techniques ground LLM outputs in external, current evidence, reducing hallucinations and improving factual accuracy [9–11]. Together, these developments open a path toward automating the full vulnerability remediation lifecycle under appropriate governance controls.

A gap persists between these lines of work. ZSM research has concentrated on network-level automation and service management, with little attention to vulnerability remediation specifically [12–15]. LLM-based security research has advanced at the code level—vulnerability detection [16,17], patch generation [18–20], and evaluation [21–23]—but no systematic designs exist for end-to-end remediation pipelines that consume scanner reports and threat intelligence across heterogeneous, multi-platform environments [2]. RAG has been applied to security question-answering and threat analysis [9,24], yet its use in operational patching workflows remains unexplored.

This study bridges that gap by integrating OpenVAS scan outputs and multi-source threat intelligence with RAG-enhanced LLM decision-making and closed-loop orchestration, applying Zero-Touch governance principles to the enterprise vulnerability management lifecycle. The resulting end-to-end remediation pipeline is designed around verifiability and auditability. Our main contributions are:

1. We propose an end-to-end Zero-Touch remediation architecture integrating OpenVAS, NVD/Cyber Threat Intelligence (CTI), RAG, and LLMs under ZSM principles. As Table 1 shows, no prior peer-reviewed work, to the best of our knowledge, combines all six capabilities in a single closed-loop pipeline: open-scanner integration, RAG-based threat intelligence retrieval, dual-LLM verification, confidence-based routing, automated CI/CD deployment with rollback, and multi-model statistical validation.
2. We design a dual-LLM verification and confidence-based routing mechanism that extends evidence-grounded approaches such as ProveRAG while reducing hallucination and erroneous patch risks.
3. We systematically evaluate how prompt engineering, input segmentation, and RAG each affect patching accuracy and latency through an empirical study on 350 real-world vulnerabilities, quantifying improvements in remediation quality, manual effort reduction, and time efficiency.
4. We implement system-level defenses and auditing mechanisms aligned with the OWASP LLM Top 10 and RAG security best practices. Reproducible implementation artifacts—including the RAG pipeline, evaluation scripts, and prompt templates—will be released upon acceptance; anonymized materials are available to reviewers upon request during the review period.

Table 1. Comparison with existing approaches.

Feature	PentestGPT [8]	ProveRAG [9]	CVE-Bench [21]	Ours
Scanner integration (OpenVAS)	No	No	No	Yes
RAG with threat intelligence	No	Yes	No	Yes
Dual-LLM verification	No	Partial (self-check)	No	Yes
Confidence-based routing	No	No	No	Yes
Automated patch deployment	No	No	Compilation test	CI/CD + rollback

Table 1. *Cont.*

Feature	PentestGPT [8]	ProveRAG [9]	CVE-Bench [21]	Ours
Multi-model evaluation	GPT-4 only	GPT-4 only	Multiple agents	5 models
Statistical validation	No	No	No	z-test; McNemar; Cohen's h
End-to-end closed-loop	No	No	No	Yes

2. Related Work

We organize our review around six areas: the vulnerability management lifecycle and Zero-Touch paradigm (Section 2.1), vulnerability scanning with emphasis on the OpenVAS ecosystem (Section 2.2), LLM applications in cybersecurity (Section 2.3), Retrieval-Augmented Generation in security workflows (Section 2.4), threat intelligence integration (Section 2.5), and prompt engineering with LLM security governance (Section 2.6). Each subsection identifies the state of the art and the gaps our framework targets.

2.1. Automated Vulnerability Management and Zero-Touch Security

Vulnerability management spans a lifecycle from asset inventory through scanning, risk assessment, patch planning and testing, deployment under change control, and post-deployment validation with feedback. NIST's National Vulnerability Database (NVD) underpins this lifecycle with structured fields—CVE identifiers, CVSS scores, CWE classifications—that enable consistent quantification and cross-tool traceability [1]. Yet as noted in Section 1, the real bottleneck is not detection: it is the timeliness and governance of patch deployment [2].

Hu et al.'s systematization of knowledge (SoK) decomposes reliable automated patching into three phases—vulnerability analysis, patch generation, and patch validation—each essential to preventing incorrect fixes and regression [22]. CVE-Bench reinforces this point empirically: real-world CVE patching depends heavily on software versioning, dependency context, and testing infrastructure, making executable validation and engineering integration non-negotiable [21].

The ETSI ISG ZSM reference architecture formalizes closed-loop autonomy through a strategy-driven perceive—analyze—decide—act model for automated, cross-domain service management [4]. Subsequent work has surveyed the ZSM landscape along several dimensions. Scope and techniques have been cataloged from intent-based networking to AI-driven orchestration in 5G/6G [15]. Cross-layer automated security solutions address authentication and intrusion detection for zero-touch 6G networks [12]. Deployment studies identify cross-domain coordination, governance, and ML-driven monitoring and resource orchestration as both key obstacles and enablers [13,14].

Benzaid et al. flag data quality, model explainability, and cross-domain closed-loop coordination as unresolved challenges when AI-driven automation meets security governance [25]. They also observe that open interfaces and embedded AI/ML widen the ZSM attack surface, demanding built-in access control, auditability, and risk isolation [26]. Rajab et al. make a complementary argument: autonomous management must enforce policy-to-enforcement consistency for sustainable closed-loop operations [27].

Several studies push Zero-Touch management into new application domains. Hazra et al. extend it to Industry 5.0, emphasizing autonomous control planes and resilient design across heterogeneous environments [28]. Jiang et al. propose a dynamic configuration framework targeting Time-Sensitive Networking (TSN) [29]. Yang et al. analyze Zero-Touch network security through AutoML, identifying reduced human involvement in model development as a deployment enabler [30]. Lira et al. explore LLM-supported Zero-Touch

configuration, concluding that natural language intent translation must be paired with thorough validation and governance [31].

Taken together, the ZSM literature supplies a conceptual and architectural foundation for closed-loop autonomy [4], while automated patching research consistently shows that generation without verification is insufficient [22]. The missing piece is a concrete implementation of closed-loop autonomy within the enterprise vulnerability management lifecycle—one that integrates scanning, threat intelligence, decision-making, deployment, and validation. Systematic designs and large-scale empirical evaluations of such integration remain scarce [2].

2.2. Vulnerability Scanning and the OpenVAS Ecosystem

Vulnerability scanners are the upstream entry point for the lifecycle described in Section 2.1. They reliably produce vulnerability inventories enriched with NVD-standardized fields—CVE, CVSS, CWE, Common Platform Enumeration (CPE) [1]—but the absence of actionable remediation decisions and deployment workflows downstream is the core operational bottleneck [2]. Without integration into validation and feedback loops, closed-loop remediation breaks down [22].

Among open-source options, OpenVAS stands out for its extensible Network Vulnerability Test (NVT) library and programmable reporting format, both well suited to heterogeneous environments and upstream automation [32]. Its interface complexity and high information density, however, raise the cost of analysis and tracking. Vimala and Fugkeaw's VAPE-BRIDGE project demonstrates that OpenVAS report fields are sufficiently structured for downstream integration by mapping them to Metasploit workflows for automated exploitability verification [33]. Still, such integrations target exploit validation, leaving a substantial gap between current capabilities and practical enterprise needs for cross-platform patch deployment and post-deployment verification [22].

Industry trends point in a similar direction. GitHub Copilot, for instance, now provides vulnerability detection and patch suggestions during development, shifting remediation leftward in the software lifecycle [34].

Yet existing solutions concentrate on container- or code-level remediation and pull request (PR)-based workflows. End-to-end automated patching that starts from host- or network-level scanner reports and targets multi-OS, multi-environment scenarios remains largely unexplored [32].

2.3. Applications of Large Language Models in Cybersecurity and Vulnerability Remediation

Systematic reviews have mapped a rapidly expanding landscape of LLM applications in cybersecurity—malware analysis, vulnerability comprehension, threat intelligence processing, incident response [6,7,35]. A consistent set of challenges recurs across these surveys: hallucination, data leakage, prompt injection, and the absence of reproducible evaluation frameworks [36–38]. In Big Data and SOC contexts, LLMs typically require integration with retrieval systems, knowledge bases, and analytics platforms to yield traceable, auditable results [39]. The consensus is clear: reliability, auditability, and governance are prerequisites before LLMs can be trusted in operational security workflows [7].

Empirical work on vulnerability detection and remediation paints a mixed picture. Fu et al. showed that ChatGPT produces useful outputs for vulnerability identification and patch suggestion, though prompt design heavily mediates effectiveness [5]. Sagodi et al. found GPT-4 still faltering on complex real-world dependencies [40]. Various approaches target specific pipeline stages: APPATCH frames patching as iterative and feedback-driven through adaptive prompting [18]; DLFix uses code transformation learning for executable fix generation [19]; Lu et al. incorporate graph structural information to sharpen vulner-

ability identification [16]; Wei et al. deploy multi-agent specialization for smart contract auditing [17]; and Bhandari et al. explore code-oriented language models for secure patch generation [20].

System-level and evaluation work adds further context. PentestGPT integrates LLMs with security toolchains via workflow-oriented task decomposition [8]. CVE-Bench tests LLM-based agents on real-world CVE patching and confirms executable validation as essential [21]. Zhou et al. survey LLM-based vulnerability detection and repair, outlining open challenges [41]. Yildiz et al. observe that real-world vulnerabilities frequently span multiple files and modules, complicating automated analysis [23]. Hu et al. argue that deploying generative patching demands validation, regression testing, and risk control [22]. These studies collectively provide a foundation for code-level remediation but leave a gap: no existing system unifies these components into a governance-aware remediation pipeline. Our work addresses this by extending the LLM's role from code patching assistant to a zero-touch remediation hub that integrates scanning, threat intelligence, and orchestration.

2.4. The Role of Retrieval-Augmented Generation (RAG) in Security and Vulnerability Management

RAG, introduced by Lewis et al. [42], combines parametric and non-parametric memory to ground language model outputs in retrieved evidence, substantially improving factual accuracy on knowledge-intensive tasks. Mend.io describes RAG as a three-stage pipeline—indexing, retrieval, generation—and argues that external knowledge sources improve traceability, thereby limiting the downstream impact of hallucinations [10]. NVIDIA emphasizes RAG's capacity to inject up-to-date external information, enhancing factual consistency [11]. OpenText analyzes RAG through an Agentic AI lens, positioning the retrieval layer as the foundation for multi-source evidence aggregation in SOC and threat investigation workflows [43].

Several cybersecurity-specific applications demonstrate RAG's value. ChatNVD combines LLMs with the NVD to support vulnerability Q&A and risk interpretation [24]. CTIBench formalizes threat intelligence tasks as benchmarkable evaluations, exposing the high hallucination risk in CTI use cases [44]. ProveRAG demonstrates that provenance and self-checking mechanisms are critical for reliable, traceable vulnerability analysis [9]. Rajapaksha et al. build a RAG-based QA system for attack investigation and attribution [45]. Telco-RAG explores retrieval and governance challenges specific to high-frequency updates on long documents [46].

RAG also introduces new attack surfaces. Shafran et al. show that blocker documents can manipulate retrieval rankings and mislead generated outputs [47]. Yao et al. identify knowledge base poisoning and sensitive data leakage as key risks in retrieval-based pipelines [36]. The OWASP LLM Top 10 flags prompt injection and sensitive data exposure as critical threats, calling for input validation, access control, and output constraints [48]. Liu et al. propose DataSentinel for systematic prompt injection detection [49]. Our framework responds to these risks by adopting source partitioning and evidence-driven retrieval when integrating OpenVAS outputs and multi-source threat intelligence, restricting the influence of low-trust data on automated patching decisions [9].

2.5. Threat Intelligence Integration and Prioritization Strategy

Effective patch prioritization demands multi-source threat intelligence (TI) beyond the static NVD risk scores discussed in Section 2.1 [1,2]. The central difficulties are heterogeneity and traceability. Intelligence sources differ in data formats, semantic granularity, and update frequency; without unified representations and provenance tracking, downstream reasoning degrades [6]. Karras et al. argue that static aggregation alone cannot produce traceable decisions and that retrieval-and-correlation mechanisms are needed [39].

CTIBench captures the diversity of CTI tasks in a benchmark framework [44], and ProveRAG shows that provenance-driven retrieval is critical for trustworthy analysis [9].

Prioritization for remediation must also reflect operational realities. Wang et al. demonstrate that real-world patching decisions hinge on software versioning, dependencies, and executable validation—not abstract CVSS scores alone [21]. Hu et al. extend this by advocating for exploitability signals and active threat intelligence as prioritization inputs under resource constraints [22]. Drawing on these insights, our framework integrates its threat intelligence layer with OpenVAS scan results within the RAG/LLM decision pipeline, enabling automatic retrieval of CTI signals during vulnerability analysis and generation of traceable, evidence-grounded prioritization justifications [9].

2.6. Prompt Engineering and LLM Security Governance

Sahoo et al. provide a systematic review of prompt engineering techniques, finding that few-shot examples, task decomposition, and structured outputs substantially influence downstream task stability and quality [50]. Chen et al. examine prompt portability and failure modes, concluding that prompt design must align with data representation and evaluation workflows [51]. Debnath et al. survey prompting methods and note that prompts often serve as the primary performance lever when labeled data is scarce [52]. Sampaio et al. warn that without verifiable output formats and governance mechanisms, prompt strategies risk producing unverifiable or misleading outputs [53].

Cybersecurity-specific work corroborates these findings. LogPrompt demonstrates that prompts can support zero-shot log analysis with interpretable outputs [54]. Zhang et al. validate prompt-based workflows for log anomaly detection [55]. Young et al. show that task-oriented prompting improves both identification and explanation in phishing detection [56].

Prompt engineering also carries security and privacy risks. Edemacu and Wu note that prompts and input data may contain sensitive information, necessitating data minimization and privacy-preserving strategies [57]. Derner et al. propose a taxonomy of prompt interaction security risks [58]. Liu et al. introduce DataSentinel for prompt injection detection [49]. Rodriguez et al. argue, from a software engineering standpoint, that prompts should be treated as governable artifacts managed through version control and regression testing [59].

The OWASP LLM Top 10 identifies prompt injection, sensitive data exposure, and overreliance on model outputs as key risks, reinforcing the need for input validation, output constraints, and auditable fallback mechanisms in automated pipelines [48]. Accordingly, our framework adopts structured output prompts and confidence-based routing, and integrates dual-LLM verification with input/output filtering to mitigate the impact of hallucinations and prompt injection on automated remediation [58].

Table 1 compares our framework against representative existing approaches across eight capability dimensions. To our knowledge, no prior work has demonstrated an open-scanner-driven, end-to-end closed-loop pipeline that jointly integrates scanning, RAG retrieval, LLM reasoning, dual verification, confidence routing, and CI/CD orchestration within a single architecture, accompanied by multi-model statistical validation. The comparison targets peer-reviewed academic systems; commercial vulnerability management platforms (e.g., Qualys VMDR, Rapid7 InsightVM, Tenable.io) offer partial automation but their proprietary architectures preclude direct feature-level comparison under the same evaluation rubric.

Six threads run through the literature and converge toward a single architectural need. ZSM supplies the governance framework for closed-loop autonomy (Section 2.1). OpenVAS provides the data substrate for scanner-driven workflows (Section 2.2). LLMs serve as

the reasoning engine (Section 2.3). RAG addresses knowledge recency and hallucination (Section 2.4). Threat intelligence feeds risk-aware prioritization (Section 2.5). Prompt engineering and security governance ensure operational safety (Section 2.6). No existing system integrates all six into a single, end-to-end pipeline with empirical validation. The next section presents our framework design.

3. System Architecture

Figure 1 illustrates the overall architecture of the proposed Zero-Touch Vulnerability Remediation System, which comprises three modular layers: the OpenVAS Scanning Layer, the AI Decision Layer, and the Orchestration Layer. Separation of concerns governs the decomposition—data acquisition, intelligent reasoning, and action execution each reside in their own layer, so any component can be scaled, tested, or replaced (e.g., substituting OpenVAS with a commercial scanner, or swapping the LLM backend) without disrupting the others. Architecturally, the three-layer pipeline maps onto the perceive—analyze—decide—act closed-loop model prescribed by the ETSI ZSM reference architecture [4].

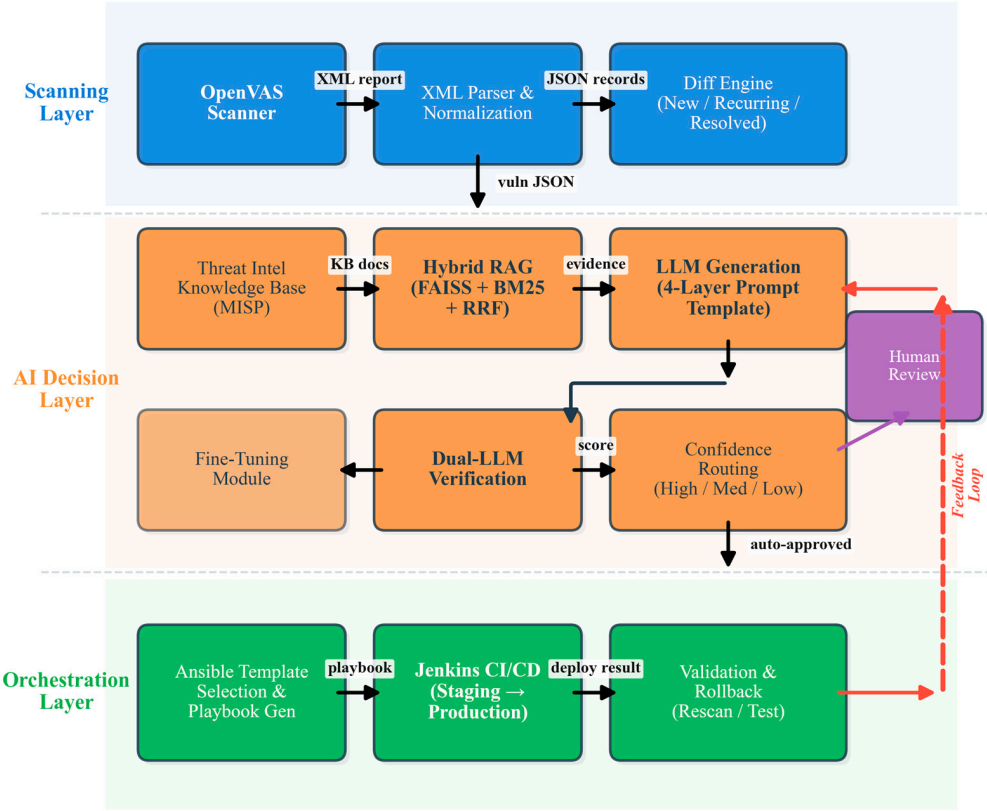


Figure 1. Overall architecture of the zero-touch vulnerability remediation system, comprising the Scanning Layer, AI Decision Layer, and Orchestration Layer. Color coding distinguishes architectural layers: blue for the Scanning Layer, orange for the AI Decision Layer, green for the Orchestration Layer, and purple for Human Review. Black arrows indicate data flow between components; the red dashed arrow represents the feedback loop. Red labels denote the data format passed at each stage.

OpenVAS periodically scans the target environment and produces vulnerability reports. The LLM engine in the AI Decision Layer analyzes these reports, drawing on multiple threat intelligence sources to generate remediation recommendations. Validated patching actions then pass to the Orchestration Layer for automated deployment and result verification. Recommendations whose confidence scores fall below the high threshold are

flagged for manual review. The following subsections detail each layer's functions and key technologies.

Algorithm 1 presents the end-to-end pseudocode of the proposed zero-touch vulnerability remediation pipeline, spanning all three architectural layers.

Algorithm 1. Zero-Touch Vulnerability Remediation Pipeline

```
Input: Target network assets T, Threat intelligence sources TI,
Confidence thresholds θ_high and θ_low, Knowledge base KB,
Top-K parameter K = 3, RRF smoothing constant c = 60
Output: Remediation execution reports R
    // === SCANNING LAYER ===
schedule OpenVAS periodic scan on T
xml_report ← OpenVAS.scan(T)
vulns ← ParseXML (xml_report) // Extract CVE, CVSS, description, etc.
for each vuln v in vulns do
v.json ← Normalize(v) // Convert to unified JSON schema
end for
delta ← DiffWithHistory (vulns, historical_DB) // New, recurring, resolved

    // === AI DECISION LAYER ===
 for each vulnerability v in vulns do
    // Step 1: RAG Retrieval
query ← BuildQuery (v.cve_id, v.description, v.service)
docs_vec ← FAISS.search (query, top_K = 2*K)
docs_kw ← BM25.search (query, top_K = 2*K)
evidence ← RRF_Merge (docs_vec, docs_kw, top_K = K)
evidence ← ApplyTemporalBoost (evidence, days = 30)

    // Step 2: LLM Generation with Structured Prompt
prompt ← BuildPrompt (SystemRole, TaskInstruction,
FewShotExamples, Scratchpad, v.json, evidence)
response ← PrimaryLLM.generate (prompt) // GPT-4-class model
recommendation ← ParseJSON (response)

    // Step 3: Dual-LLM Verification (AuxLLM = gpt-4o)
verification ← AuxLLM.verify (recommendation, evidence)
confidence ← ComputeConfidence (verification)

    // Step 4: Confidence-Based Routing
if confidence >= θ_high then
 route v to ORCHESTRATION_LAYER (auto-execute)
else if confidence >= θ_low then
route v to HUMAN_REVIEW_QUEUE (semi-automated)
else
route v to MANUAL_HANDLING
 end if
end for

    // === ORCHESTRATION LAYER ===
```

Algorithm 1. *Count.*

```
for each auto-approved recommendation rec do
playbook ← SelectAnsibleTemplate (rec.action_type)
script ← FillTemplate (playbook, rec.details)
snapshot ← CreateSnapshot (rec.target_host)
result ← Jenkins.deploy (script, environment = "staging")
if ValidateDeployment (result) == SUCCESS then
Jenkins.promote (script, environment = "production")
 else
Rollback (snapshot)
FlagForManualInvestigation (rec)
end if
   // Feedback loop
UpdateKB (KB, rec, result) // Reinforce or penalize strategy
R.append (GenerateReport (rec, result))
end for
return R
```

3.1. Scanning Layer

OpenVAS serves as the core scanner in this layer, relying on its extensive plugin library to conduct periodic scans of target network assets. For each scan result, the system parses the OpenVAS-generated XML reports and extracts structured fields—CVE ID, description, severity, CVSS score, affected scope, and any initial mitigation suggestions that OpenVAS provides. All extracted data is normalized into a unified JSON format before being forwarded to the AI Decision Layer.

An event-driven triggering mechanism supplements the batch workflow: upon detecting a new high-risk or known exploitable vulnerability, the layer immediately forwards the relevant data to the AI Decision Layer for prioritized analysis, bypassing the wait for full report parsing. Historical scan results also undergo audit comparisons to identify new, recurring, or disappeared vulnerabilities; these deltas feed into the decision layer for trend analysis and risk tracking.

Figure 2 illustrates the JSON schema used for normalizing OpenVAS scan results.

3.2. AI Decision Layer

Five components constitute this layer: the Threat Intelligence Knowledge Base, the Hybrid RAG Retrieval Engine, the LLM Generation Module, the Dual-LLM Verification unit, and the Confidence-Based Routing mechanism. Figure 3 presents the internal architecture and data flow among these components; Figure 4 provides a step-by-step process flowchart tracing the decision pipeline from vulnerability input to routing output. Together, these components transform raw vulnerability records from the Scanning Layer into concrete, evidence-grounded remediation recommendations.

```
{
  "cve_id": "CVE-2024-12345",
  "description": "Buffer overflow in libxml2 before 2.12.6...",
  "cvss_score": 9.8,
  "severity": "Critical",
  "affected_hosts": ["192.168.1.10", "192.168.1.22"],
  "service_name": "Apache/2.4.51",
  "port": 443,
  "openvas_remediation_hint": "Update to libxml2 >= 2.12.6",
  "scan_timestamp": "2025-01-15T08:30:00Z",
  "nvt_oid": "1.3.6.1.4.1.25623.1.0.170001"
}
```


Figure 2. Normalized JSON schema for OpenVAS vulnerability records. Each record contains fields for 'cve_id', 'description', 'cvss_score', 'severity', 'affected_hosts', 'service_name', 'port', and 'openvas_remediation_hint'. Note: CVE identifiers shown are illustrative placeholders.

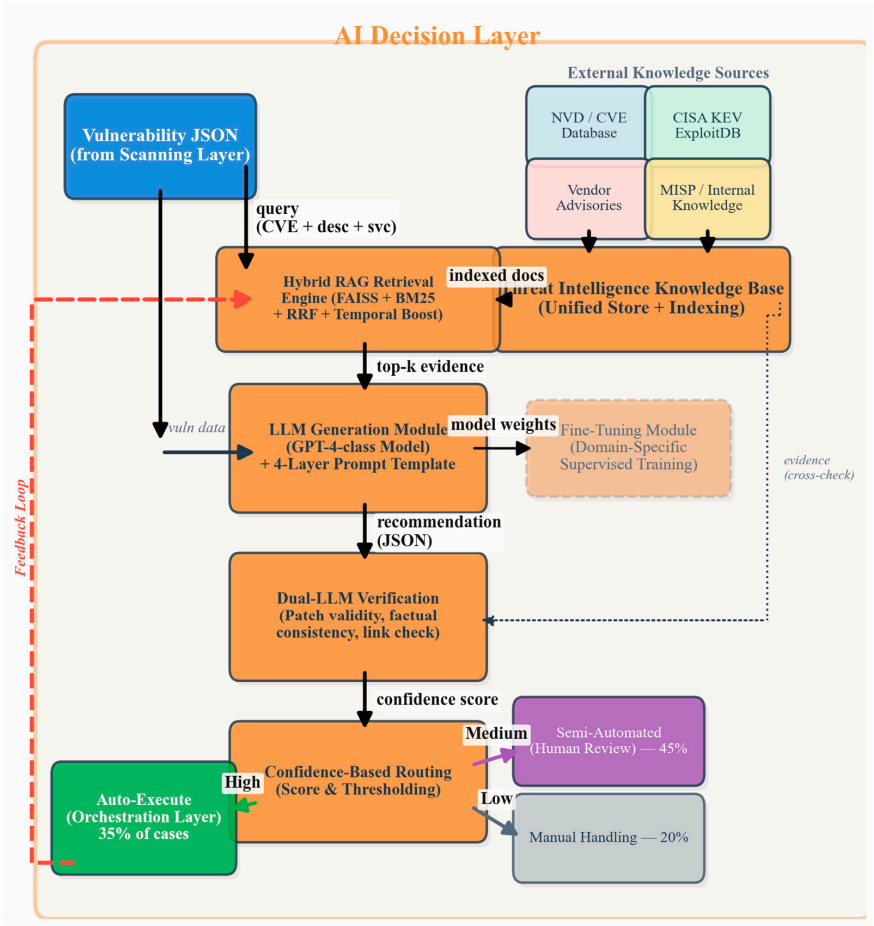


Figure 3. Internal architecture of the AI Decision Layer, showing the data flow among the Threat Intelligence Knowledge Base, Hybrid RAG Engine, LLM Generation Module, Dual-LLM Verification, and Confidence-Based Routing. Orange boxes represent core processing modules; the blue box denotes the input from the Scanning Layer; green indicates the auto-execution path; purple indicates semi-automated human review; gray indicates fully manual handling. Pastel-colored boxes (top right) represent external knowledge sources. Solid arrows show data flow; the dotted arrow indicates evidence cross-checking; the red dashed line represents the feedback loop.

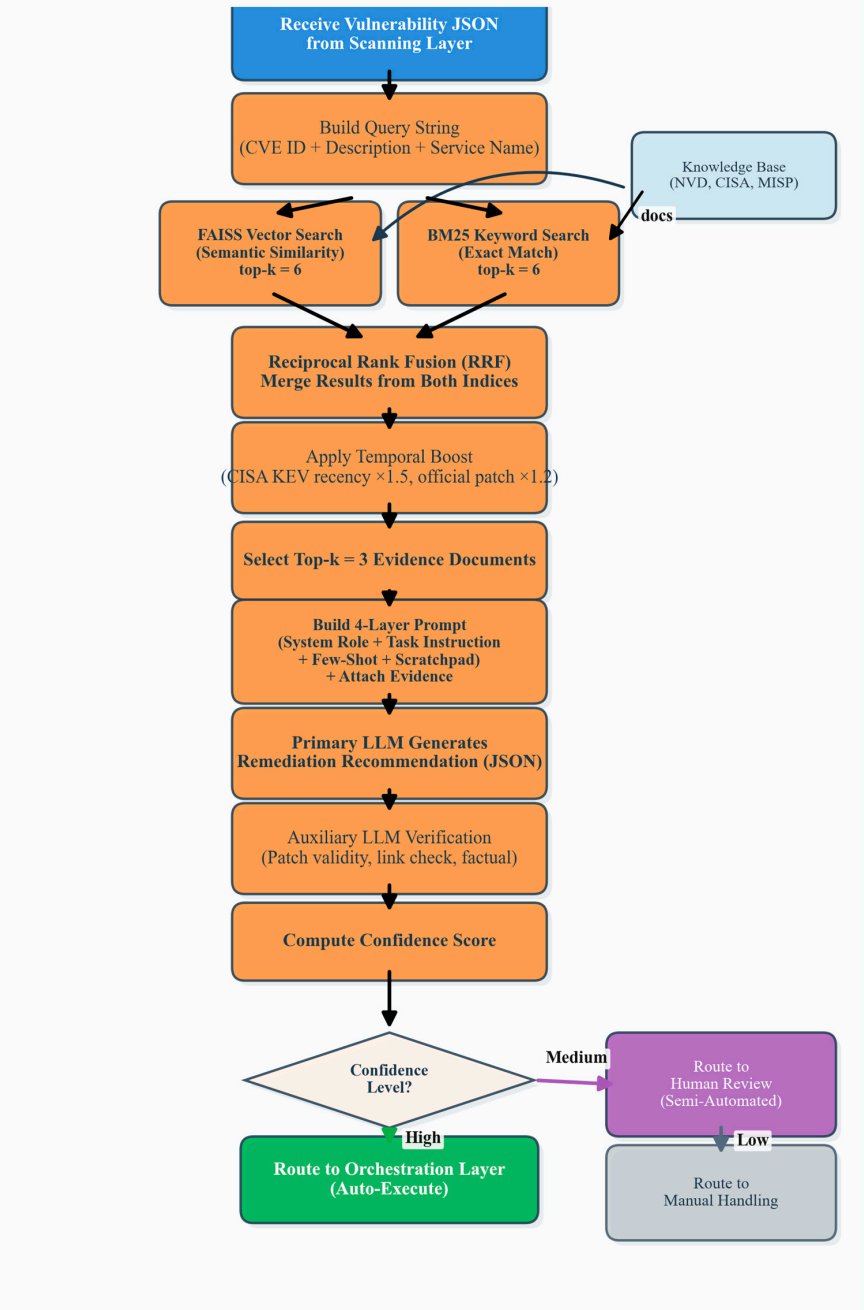


Figure 4. Process flowchart of the AI Decision Layer, illustrating the step-by-step pipeline from vulnerability input through hybrid retrieval, prompt construction, LLM generation, dual verification, and confidence-based routing. Orange boxes denote sequential processing steps; the blue box marks the pipeline entry point; the diamond represents the confidence-level decision node. Green indicates the high-confidence auto-execution path, purple the medium-confidence human-review path, and gray the low-confidence manual-handling path. Arrows indicate processing flow direction.

This layer comprises the following components:

(1) Threat Intelligence Aggregation and Knowledge Base: We build a unified local vulnerability knowledge base centered around the Malware Information Sharing Platform (MISP), integrating threat intelligence and patch data from multiple sources. For each vulnerability, the knowledge base contains descriptions, root cause analyses, known exploitation status, vendor advisories, and corresponding patches or mitigation measures. To improve retrieval efficiency, textual entries are preprocessed and normalized (e.g., unifying field names, removing duplicates), and a hybrid retrieval system is implemented using both

Facebook AI Similarity Search (FAISS) vector indexing and Best Match 25 (BM25) keyword indexing. Text embeddings are generated using OpenAI's text-embedding-3-small model (1536 dimensions); each KB entry is chunked into segments of up to 512 tokens with a 64-token overlap to preserve cross-boundary context. The FAISS index uses inner-product similarity, while BM25 uses default Okapi parameters (k_1 = 1.2, b = 0.75). During retrieval, both indices return 2K candidates (with K = 3 as the final top-K), which are fused via Reciprocal Rank Fusion (RRF) with smoothing constant c = 60. We adopt a fixed top-K strategy (K = 3) rather than a similarity-threshold cutoff because vulnerability descriptions vary widely in embedding-space density; a fixed K guarantees a bounded prompt length and predictable latency, while threshold-based filtering risks returning zero or excessively many results for atypical queries. The choice of K = 3 was determined by a grid search over K ∈ {1, 3, 5, 7} on the 50-case validation set, where K = 3 maximized accuracy while keeping mean input tokens below 1000, and optionally boosted by temporal recency (window W = 30 days; see Algorithm 2). The temporal window W = 30 was chosen empirically: vulnerability exploitability is highest in the first month after public disclosure, aligning with CISA's remediation guidance timelines. Similarly, the source trustworthiness boost factors (α_auth = 1.3 for authoritative, α_recent = 1.5 for recent KEV/Exploit-DB entries with official patches receiving an additional α_patch = 1.2; see Algorithm 2) were selected based on preliminary experiments on a held-out validation set of 50 cases. Systematic sensitivity analysis across parameter ranges (e.g., W ∈ {7, 14, 30, 60, 90} days; boost factors α ∈ {1.1, 1.2, 1.3, 1.5}) is planned as future work to characterize the accuracy–recency trade-off and guide deployment tuning. This enables the system to perform semantic similarity searches while also supporting exact keyword matches (e.g., specific CVE IDs or product names).

Knowledge base ingestion and governance. The KB is populated through an automated ingestion pipeline operating on a configurable schedule: NVD and CISA KEV feeds are polled every 6 h; vendor advisory RSS feeds (e.g., Red Hat, Microsoft, Canonical) are ingested daily; and commercial CTI subscriptions are synchronized upon push notification. Each incoming document undergoes: (i) deduplication via CVE-ID and content hash; (ii) source trustworthiness tagging on a three-tier scale: authoritative (NVD, CISA KEV, vendor security advisories from Red Hat, Microsoft, Canonical, and Apache), community (Exploit-DB, GitHub Security Advisories, MISP community feeds), and unverified (blog posts, forum discussions, unvetted RSS). During retrieval, authoritative sources receive a 1.3× relevance boost (α_auth = 1.3); unverified sources are surfaced only when no higher-tier evidence is available, and are flagged with a provenance warning in the LLM prompt. When contradictory advisories exist (e.g., differing patch versions from vendor vs. community sources), the system prioritizes the higher-trust source and appends both references for human review. This tagging is maintained via a manually curated source whitelist; (iii) schema normalization to the unified JSON format; and (iv) embedding generation for FAISS indexing. Stale entries are flagged—not deleted—when superseded by newer advisories, preserving audit trails. At the time of our experiments, the KB contained 4305 entries spanning 15 vulnerability categories. Access to the KB is governed by role-based access controls, and all ingestion events are logged to an immutable audit trail for compliance purposes.

(2) RAG Retrieval Engine: Upon receiving the structured vulnerability JSON, the AI Decision Layer initiates a retrieval process based on key fields such as CVE ID, vulnerability description, and service name. Algorithm 2 presents the hybrid retrieval procedure.

Algorithm 2. Hybrid RAG Retrieval with Temporal Boosting

```
Input: Vulnerability record v, Vector index VI (FAISS),
Keyword index KI (BM25), Knowledge base KB,
Top-K parameter K = 3, RRF smoothing constant c = 60, Temporal window W = 30 days,
       Source boost factors α_auth = 1.3, α_recent = 1.5, α_patch = 1.2
Output: Ranked evidence documents E
query_text ← Concat(v.cve_id, v.description, v.service_name)
query_vec ← EmbeddingModel.encode(query_text)

    // Parallel retrieval from both indices
results_vec ← VI.search(query_vec, top_K = 2*K)
results_kw ← KI.search(query_text, top_K = 2*K)

    // Reciprocal Rank Fusion (RRF)
merged ← {}
for each doc d in results_vec ∪ results_kw do
rank_v ← rank of d in results_vec (or infinity if absent)
rank_k ← rank of d in results_kw (or infinity if absent)
merged[d].score ← 1/(c + rank_v) + 1/(c + rank_k)
end for

    // Temporal relevance boosting
for each doc d in merged do
if d.source in {"CISA_KEV", "ExploitDB"} then
if d.last_exploited_date >= (today − W) then
merged[d].score ← merged[d].score × α_recent    // α_recent = 1.5
end if
end if
if d.has_official_patch == true then
merged[d].score ← merged[d].score × α_patch    // α_patch = 1.2
end if
end for

// Source trustworthiness boosting
for each doc d in merged do
    if d.source_tier == "authoritative" then
      merged[d].score ← merged[d].score × α_auth    // α_auth = 1.3
    end if
end for
E ← TopK (merged, K)
return E
```

Three design choices in Algorithm 2 merit justification. First, we adopt a hybrid retrieval strategy combining FAISS (dense vector search) with BM25 (sparse keyword matching) because vulnerability queries exhibit a dual nature: semantic similarity captures conceptually related advisories, while exact keyword matching is essential for precise identifiers such as CVE IDs, package names, and version strings. Neither modality alone achieves sufficient recall across both query types. Second, we fuse the two result lists using Reciprocal Rank Fusion (RRF) rather than learned score combination because RRF is a lightweight fusion method with a single hyperparameter (c = 60 in our implementation),

avoids learning score calibration across heterogeneous rankers, and requires no training data—an advantage when the knowledge base evolves continuously [60]. Third, we apply temporal relevance boosting (lines 17–26) and source trustworthiness boosting (lines 28–33) because vulnerability exploitability is time-sensitive: recently weaponized CVEs listed in the CISA KEV catalog or Exploit-DB pose disproportionately higher risk, and entries with official vendor patches should be surfaced preferentially to enable definitive remediation over temporary workarounds.

In our experiments, each query returns approximately three highly relevant text segments that describe the vulnerability's background, patch details, or similar known cases.

(3) LLM Generation Module: A GPT-4-class large language model serves as the core generation engine. Specialized prompt templates specify the task description, expected input patterns, and output formatting requirements—including a structured JSON schema. Each prompt bundles the OpenVAS scan summary alongside the evidence fragments retrieved during the RAG step, grounding the model's output in referenced, authoritative knowledge. The resulting JSON structure contains proposed remediation actions (e.g., installing specific patch versions, adjusting configuration settings) and root cause analysis.

(4) Auxiliary Validation and Routing: To ensure the accuracy and feasibility of the generated remediation recommendation, the AI Decision Layer initiates a secondary validation stage following the primary model's output. The rationale for dual-LLM verification is analogous to the "four-eyes principle" in financial auditing: a single generative model may hallucinate plausible but incorrect patch versions or fabricate URLs, and self-consistency checks within the same model are insufficient because the model shares identical biases [9]. By delegating verification to a separate auxiliary LLM—gpt-4o in all experiments reported here—the system introduces an orthogonal validation perspective. The auxiliary verifier checks whether referenced patch versions exist in the knowledge base and whether the recommended action is consistent with the retrieved evidence. It then assigns a confidence score to the generated recommendation. We implement a three-tier routing scheme: if the confidence score exceeds a high threshold ($\theta_high = 0.85$), the patching plan is forwarded to the Orchestration Layer for fully automated execution; if the score falls in the intermediate range ($\theta_low \leq$ confidence $< \theta_high$), the case is queued for human review with a pre-filled recommendation to accelerate analyst decision-making; below θ_low, the case is routed to manual handling. This confidence-based routing mechanism minimizes the risk of incorrect remediation while maximizing automation throughput, striking a balance between efficiency and operational safety. In our pilot deployment, we set $\theta_high = 0.85$ and $\theta_low = 0.60$, yielding the 34.9/45.1/20.0 triage distribution reported in Section 4.2. These thresholds were calibrated on a held-out validation subset (50 cases) by requiring that the rollback rate remain below 5% while routing at least 30% of cases to automation; θ_low was set to capture the inflection point where analyst review time savings outweighed the cost of manual triage. A sensitivity analysis of $\theta_high \in \{0.80, 0.85, 0.90\}$ with θ_low fixed at 0.60 is reported in Section 5.4, confirming that $\theta_high = 0.85$ is operationally equivalent to 0.90 under the current discrete scoring scheme. "Successfully patched and verified" is defined as: (a) the Ansible playbook executed without non-zero exit codes; (b) a post-deployment health check (HTTP 2xx, process liveness, and basic regression test) passed within a 10min observation window; and (c) an OpenVAS re-scan confirmed that the target CVE is no longer reported. Among the 122 high-confidence cases routed to auto-execution, 5 cases (5/122 = 4.1%, the "audit-flag rate") were later found upon manual audit to contain suboptimal but non-harmful actions (e.g., upgrading to a newer-than-necessary version); no false auto-executions caused service degradation or security regressions. (This audit-flag rate—5/122 = 4.1%—should not be confused with the automated rollback rate of 4/97 = 4.1% reported in the pilot-deployment metrics, which captures a distinct failure

mode: infrastructure-level deployment issues detected by post-deployment health checks. The near-identical percentages are coincidental, arising from different numerators and denominators.) The current thresholds were empirically tuned on our dataset; extending the sensitivity analysis to θ_low variation and multi-site environments remains future work.

The dual-LLM verification procedure is formalized in Algorithm 3. The auxiliary verifier (gpt-4o) receives the primary model's recommendation alongside the retrieved evidence and applies four independent checks, each producing a binary signal that is combined into a weighted confidence score.

Algorithm 3. Dual-LLM Verification Procedure

Input: recommendation R from primary LLM, retrieved evidence E, knowledge base KB
Output: confidence score c ∈ [0, 1], verification verdict v ∈ {PASS, FAIL}
function DUAL_LLM_VERIFY (R, E, KB)
    // Check 1: Version existence
    v_exists ← 1 if R.patch_version ∈ KB.known_versions else 0
    // Check 2: Evidence alignment
    e_aligned ← 1 if SEMANTIC_SIM (R.rationale, E) > 0.7 else 0
    // Check 3: Action feasibility
    a_feasible ← 1 if R.action_type ∈ ALLOWED_ACTIONS else 0
    // Check 4: Format validity
    f_valid ← 1 if VALIDATE_JSON_SCHEMA(R) else 0
    // Weighted confidence computation
    c ← 0.3 · v_exists + 0.4 · e_aligned + 0.2 · a_feasible + 0.1 · f_valid
    v ← PASS if (c ≥ θ_high ∧ f_valid = 1) else FAIL
    return (c, v)
end function

The confidence score c is computed as a weighted linear combination: $c = 0.3 \cdot v_exists + 0.4 \cdot e_aligned + 0.2 \cdot a_feasible + 0.1 \cdot f_valid$, where each indicator is binary (0 or 1). The weights reflect operational priorities: evidence alignment receives the largest weight (0.4) because hallucinated content that contradicts retrieved evidence poses the greatest risk; version existence (0.3) catches fabricated patch identifiers; action feasibility (0.2) enforces the policy whitelist; and format validity (0.1) ensures downstream parsability. The weights were determined by logistic regression on the 50-case validation set and remained fixed for all 350 test cases. A case scores c = 1.0 only when all four checks pass; any single failure reduces c by at least 0.1, typically routing the case to human review. In addition, outputs that fail JSON schema validation (f_valid = 0) are unconditionally excluded from the automated execution tier regardless of the aggregate score; this hard gate ensures that no structurally malformed recommendation reaches the Orchestration Layer, even in the edge case where the remaining three indicators would yield $c \geq \theta_high$.

The verification prompt sent to the auxiliary LLM (gpt-4o) follows a structured template: the system message defines the verifier role and output JSON schema ({"verdict": "PASS | FAIL", "confidence": float, "issues": [str]}); the user message presents the primary model's recommendation, the top-K retrieved evidence passages, and explicit instructions to check version existence against the KB, evidence alignment, action feasibility, and output format. The structured output constraint ensures deterministic parsing and enables automated routing without post-hoc extraction.

(5) Model Fine-Tuning: In certain highly specialized or complex scenarios, prompt engineering and retrieval alone proved insufficient for general-purpose LLMs to produce optimal outputs. A fine-tuning module addresses this gap through targeted model en-

hancement. Expert-written Q&A pairs and high-quality patch analysis documents were collected to perform domain-specific fine-tuning on gpt-3.5-turbo. After fine-tuning, the model achieved strong performance in our Prompt + RAG evaluation, with improved remediation specificity and contextual alignment.

3.3. Orchestration Layer

Once validated, remediation recommendations pass to the Orchestration Layer, which translates them into executable automation playbooks (workflow scripts) and closes the loop from deployment through post-deployment verification.

Jenkins handles continuous integration/deployment (CI/CD), while Ansible provides agentless, idempotent configuration management across heterogeneous OS environments (Linux, Windows). Jenkins was selected for its mature plugin ecosystem and widespread enterprise adoption; Ansible for its suitability to infrastructure automation without persistent agents. Execution proceeds as follows. First, the system identifies required action types (e.g., package updates, configuration changes) from the remediation recommendation and selects the corresponding prebuilt Ansible templates. Vulnerability-specific details populate the template to produce a customized deployment script. Jenkins then pushes the script to the affected target system. A staged deployment strategy governs rollout—changes are first applied in a staging environment that mirrors production topology. Only after validation tests pass (service health checks, regression tests, and re-scanning) does the patch promote to production. This canary-style progression confines the blast radius of any faulty patch to the staging environment, where rollback cost is minimal.

Post-deployment, re-scan operations or predefined validation tests confirm whether the vulnerability has been remediated and associated services remain functional. Failed validations trigger an automated rollback to the pre-change system state, and the case is flagged for manual investigation. All logs, execution results, and potential errors are recorded and surfaced through reports and dashboards for audit purposes.

This layer also supports continuous learning and feedback: after each remediation task, the system sends execution data (success/failure, time taken, dependency issues) back to the AI Decision Layer's knowledge base:

- Successful cases reinforce the reliability score of the corresponding remediation strategy.
- Failed or rolled-back cases trigger updates to the knowledge base, for example, documenting compatibility issues between a specific patch and OS version.

Over time, this feedback loop can support reinforcement learning or rule updates that progressively sharpen the system's decision-making accuracy.

3.4. Prompt Engineering and Knowledge Fusion Techniques

Prompt engineering and input optimization received substantial attention during development. The final prompt template follows a four-layer structure:

1. System Role: Instructs the model to act as a senior vulnerability analyst, ensuring a professional and rigorous tone. It also explicitly directs the model to follow a specific output format (i.e., a JSON structure) and not to reveal the reasoning process to the end user.
2. Task Instruction: Describes the definition of input and output fields. The model is instructed to parse an <Input_XML> containing OpenVAS report data and extract fields such as id, description, root_cause, attack_vector, cvss_score, severity, and remediation, outputting them in JSON format.
3. Few-Shot Examples: We manually crafted several representative examples, including common vulnerabilities and cases with missing information. These examples are embedded in the prompt, showcasing both input samples and expected output formats.

4. Output Constraints: We added an [Output Constraints] section that explicitly enforces schema compliance: all fields are required (with null for insufficient evidence), and the model is forbidden from fabricating CVE identifiers, package versions, or URLs. This section serves as a guardrail against hallucinated artifacts—the most dangerous error category in automated remediation.

Figure 5 presents a concrete example of the four-layer prompt template used in our system.

```
[SYSTEM ROLE]
You are a senior cybersecurity vulnerability analyst.
Your task is to analyze OpenVAS scan results and generate
structured remediation recommendations in JSON format.
Follow CVSS v3.1 scoring conventions.

[TASK INSTRUCTION]
Given the <Input_XML>, extract and output a JSON object:
  - id: CVE identifier
  - description: vulnerability summary
  - root_cause: underlying technical cause
  - attack_vector: Network/Adjacent/Local/Physical
  - cvss_score: float (0.0-10.0)
  - severity: Critical/High/Medium/Low
  - remediation: actionable fix recommendation

[FEW-SHOT EXAMPLES]
Example 1:
  Input:  CVE-2021-44228 (Log4Shell), CVSS 10.0
  Output: {"remediation": "Upgrade log4j to >= 2.17.1"}
Example 2:
  Input:  CVE-2024-XXXX (Unknown), CVSS N/A
  Output: {"remediation": "Insufficient data. Flag for
           manual review.", "confidence": 0.2}

[OUTPUT CONSTRAINTS]
- Return ONLY valid JSON conforming to schema above.
- Every field REQUIRED; use null if insufficient evidence.
- Do NOT fabricate CVE IDs, versions, or URLs.
```

Figure 5. Illustrative example of the four-layer prompt template used for vulnerability remediation generation. Color blocks indicate the four prompt layers: blue for system role, orange for task instruction, green for few-shot examples, and purple for output constraints.

OpenVAS reports are often lengthy and structurally complex. A long-text segmentation strategy addresses this: each report is decomposed into individual vulnerability entries that are fed to the model in batches, preventing context-length overflows and reducing parsing complexity. Batch outputs are later aggregated into a complete remediation plan.

Combining prompt optimization with segmented processing improved baseline output quality without modifying model weights. Where prompt tuning alone remains insufficient—particularly in high-sensitivity scenarios—supervised fine-tuning provides an additional enhancement mechanism. The fine-tuning dataset comprised approximately 2000 question-answer pairs built through a two-stage process: (1) automated synthesis using the RAG pipeline to generate candidate Q&A pairs from real vulnerability records, and (2) expert curation, where two domain specialists reviewed, corrected, and filtered the candidates for factual accuracy and remediation specificity. The resulting dataset covers 15 vulnerability categories across four platform categories (Linux, Windows, macOS, and container images).

Fine-tuning was applied exclusively to GPT-3.5-turbo using the OpenAI fine-tuning API. The training objective was constrained to the output format compliance layer—penalizing non-conforming JSON structures and hallucinated package versions—while preserving the model's general language and reasoning capacity. Training hyperparameters were: 3 epochs, batch size 4, learning rate multiplier 0.1 (OpenAI default scaling), with a 90/10 training/validation split. Early stopping was monitored on validation loss with a patience of 1 epoch. The fine-tuning job completed in approximately 45 min on OpenAI's managed infrastructure. Training examples were formatted in the OpenAI chat-completion fine-tuning schema with system, user, and assistant roles to match the inference-time prompt structure, ensuring that the fine-tuned model learns both the reasoning pattern and the structured JSON output format expected during deployment.

Fine-tuning data isolation. To prevent information leakage between the fine-tuning set and the 350-case evaluation set, we enforced CVE-level deduplication: all CVE IDs appearing in the evaluation set were excluded from the fine-tuning corpus prior to training. Specifically, the 2000 Q&A pairs were derived from vulnerability records collected during the first eight months of the 12-month collection period, while the 350 evaluation cases were drawn from the final quarter (as described in Section 4.1). We further verified that no fine-tuning example shared a (CVE-ID, platform) tuple with any evaluation case. This temporal and identifier-based separation ensures that the fine-tuned model's performance gains reflect genuine domain adaptation rather than memorization of test answers.

After fine-tuning, the model reached 81.6% accuracy under the Prompt + RAG configuration—matching the larger gpt-4.1 (82.0%)—demonstrating that targeted fine-tuning on curated domain data can close the gap between smaller and frontier models (see Section 5.1 for further discussion).

Collectively, these strategies yield a knowledge-grounded, prompt-optimized workflow for vulnerability analysis and remediation generation. The model retains its general language capabilities while anchoring outputs in external authoritative evidence.

4. Experimental Design and Results

We now turn to evaluating the framework on real-world enterprise data. Section 4.1 describes the dataset and experimental setup; Section 4.2 presents results and analysis.

4.1. Dataset and Experimental Setup

Dataset. The evaluation dataset consists of 350 manually verified vulnerability records drawn from OpenVAS scan reports collected over 12 months from an enterprise internal network. The records span multiple operating systems and application software, with CVSS v3.1 severity scores between 4.0 (medium) and 9.8 (critical). For each vulnerability, the enterprise security team's actual remediation action—crafted and implemented by domain experts—serves as the gold standard.

The 350 cases break down as follows. By CVSS severity: 38% Critical ($\geq$9.0), 45% High (7.0–8.9), 17% Medium (4.0–6.9). By vulnerability category, the five most common types are remote code execution (22%), privilege escalation (18%), denial of service (15%), information disclosure (14%), and authentication bypass (11%); the remaining 20% includes cross-site scripting, SQL injection, buffer overflow, and configuration weaknesses. By platform: Linux (52%), Windows (28%), container images (12%), macOS (8%).

Two cybersecurity domain experts independently rated model-generated remediation recommendations against the gold standard, achieving a Cohen's kappa of 0.87. The dataset spans diverse vulnerability types and affected environments, including web services, databases, middleware, and network devices.

Target Models. We evaluated five models from the GPT family: gpt-4.1, gpt-4o-mini, gpt-3.5-turbo, gpt-4o, and gpt-o4-mini. Of these, gpt-4o is OpenAI's optimized multimodal model, and gpt-o4-mini is a compact reasoning-focused model from the o-series. Model names (e.g., gpt-4o-mini, gpt-o4-mini, gpt-4.1) follow Azure OpenAI Service API deployment conventions at the time of experiments; readers should consult current documentation for the latest identifiers. Only gpt-3.5-turbo was fine-tuned (Section 3.4); all other models were tested via real-time API access without weight modification, using identical prompt templates and the same retrieval-augmented knowledge base.

Experimental Setup. We designed three configurations to isolate the contribution of each technique:

- Baseline: Basic prompt only. The model receives the raw OpenVAS report using a minimal prompt without advanced prompt engineering or RAG-based retrieval. This is the starting point representing unoptimized LLM performance.
- Prompt: Prompt-optimized only. This configuration uses our four-layer prompt template along with field extraction and long-text segmentation, but excludes RAG retrieval. It isolates the performance gain attributable to prompt engineering alone.
- Prompt + RAG (Final): Full system configuration. This activates all components: prompt optimization, long-text segmentation, and RAG-based threat intelligence retrieval. For gpt-3.5-turbo, all three configurations in the comparative performance results reported in Section 4.2 were evaluated using the fine-tuned variant (see Section 3.4); thus the Baseline and Prompt columns isolate the effects of prompt engineering and RAG retrieval within the fine-tuned model, rather than comparing against the original (non-fine-tuned) base model. All other models were evaluated without weight modification. It represents the fully realized version of the proposed solution. Throughout the remainder of this paper, we use "Final" and "Prompt + RAG" interchangeably to refer to this configuration.

Non-LLM baseline configurations. To isolate the contribution of LLM generation from retrieval alone, we define two non-LLM baselines below. Preliminary results on a 50-case subset are reported in the non-LLM baseline comparison in Section 4.2; full protocols are specified here for transparency and replication using the artifacts released upon acceptance (see Data Availability):

- OpenVAS-Hint Baseline: For each case in the 50-case evaluation subset (see the non-LLM baseline comparison in Section 4.2), the openvas_remediation_hint field from the scanner output (see Figure 2) is extracted and formatted into the same JSON output schema used by the LLM configurations. Fields that the hint does not cover (e.g., root_cause, attack_vector) are set to null. The resulting outputs are scored against the gold standard using the identical evaluation rubric and dual-rater protocol. This baseline establishes the scanner-only lower bound—i.e., what an organization would achieve by acting solely on scanner-provided guidance without any LLM or retrieval augmentation.
- Retrieval-Only Baseline: For each of the same 50 evaluation-subset cases, the hybrid RAG retrieval engine (Algorithm 2) is executed identically to the Prompt + RAG configuration, returning the top-K (K = 3) evidence documents. Instead of passing these to an LLM, the system concatenates the retrieved documents and applies a deterministic template-based extractor to populate the output JSON schema: the remediation field is filled with the first actionable sentence from the highest-ranked advisory; root_cause and attack_vector are extracted via regex patterns matching CVSS vector strings and CWE descriptions; remaining fields are populated from the vulnerability record metadata. Outputs are truncated to match the LLM output length and scored using the same rubric. This baseline isolates the value of retrieval

infrastructure from generative synthesis, directly addressing whether the accuracy gains are attributable to RAG retrieval alone or to the LLM's ability to synthesize cross-document evidence into coherent remediation plans.

Under each configuration, all models generated remediation recommendations for the full 350 cases. Outputs were compared against gold-standard solutions using the following metrics:

- Accuracy (primary metric): The proportion of cases in which the model's recommended remediation actions were judged correct by both expert raters. A recommendation was rated correct if it satisfied all of the following criteria: (a) the primary remediation action type matched the gold standard (e.g., package upgrade, configuration change, service restart); (b) the target software component was correctly identified; and (c) the recommended version or patch identifier was either an exact match or a functionally equivalent alternative (e.g., recommending version 2.12.7 when the gold standard specifies $\geq$ 2.12.6). Multi-step remediations were scored as correct only if all critical steps were present. Cases where the two raters disagreed were resolved through discussion until consensus was reached. We note two additional disambiguation rules: (i) when multiple valid remediation paths existed for a single vulnerability (e.g., upgrading to version A or applying vendor workaround B), the recommendation was scored correct if it matched any accepted alternative documented by both raters during gold-standard construction; (ii) recommendations that proposed a strictly newer patch version than the gold standard were accepted as correct provided the raters confirmed the newer version also addresses the target CVE, reflecting the practical reality that vendor-recommended versions may evolve between gold-standard creation and evaluation.
- Hallucination rate: The proportion of recommendations containing at least one factually unsupported or verifiably incorrect claim, including fabricated CVE identifiers, non-existent software versions, or remediation steps contradicted by vendor advisories. Each recommendation was independently assessed by both raters against the knowledge base and vendor documentation.
- Knowledge coverage rate: The proportion of recommendations that cite at least one retrieved evidence segment relevant to the target CVE, as judged by rater consensus.
- Latency: Average time from input submission to response generation.
- Token Usage and Accuracy per 1K Tokens (acc/1kTok = Accuracy $\times$ 1000/Average Tokens per Query): Cost-effectiveness metric.

All experiments were repeated three times under identical conditions; averaged results are reported. Because the same 350 cases are evaluated across configurations, Two-proportion z-tests are reported as the primary cross-model summary in Section 4.2; McNemar's test on paired per-case outcomes and bootstrap confidence intervals are additionally reported as robustness checks (Section 4.2).

Inference settings and reproducibility. Unless otherwise specified, all models were evaluated with temperature = 0, top_p = 1.0, and a maximum output length of 2048 tokens, using the same system prompt and JSON schema constraints across all configurations. Latency was measured end-to-end, including RAG retrieval time and model generation time, from request submission to receipt of a valid JSON output. Token usage reports the sum of input and output tokens. All experiments were conducted between October and December 2025 using the same knowledge base snapshot (4305 entries) and identical software versions to ensure reproducibility. The four non-fine-tuned models were accessed via the Azure OpenAI Service API with date-pinned deployments (e.g., gpt-4o-2024-08-06, gpt-4o-mini-2024-07-18) to prevent silent model updates during the evaluation period. The system prompt was version-controlled and remained unchanged across all runs. API

calls were configured with a 60 s timeout and exponential-backoff retry (up to 3 attempts) to handle transient rate-limit or network errors; failed requests after retries were logged and excluded from latency statistics (fewer than 0.5% of total calls). The knowledge base snapshot used for evaluation was generated as a full export; in production, the ingestion pipeline operates incrementally (NVD/CISA KEV polled every 6 h, vendor RSS daily) as described in Section 3.2.

Hardware and software environment. Standard models (gpt-4.1, gpt-4o, gpt-4o-mini, gpt-o4-mini) were accessed via the Azure OpenAI Service API for inference; the fine-tuned gpt-3.5-turbo was trained and served through OpenAI's native fine-tuning API, as Azure OpenAI did not support fine-tuning for this model at the time of experiments. No local GPU resources were required for inference; all models ran as cloud-hosted endpoints. The RAG retrieval pipeline (FAISS indexing, BM25 scoring, and RRF) ran on a single CPU node (Intel Xeon E5-2680 v4, 64 GB RAM) and completed knowledge base indexing of 4305 entries in approximately 2 min. A single pass of the full evaluation campaign (five models, three configurations, 350 cases each, sequential API calls) required approximately 5–6 h of wall-clock time; reported metrics were averaged over three such passes, dominated by LLM inference latency—in particular gpt-o4-mini at 16.78 s per case under Prompt + RAG. Fine-tuning of gpt-3.5-turbo consumed approximately 45 min for 3 epochs on the ~2000-example curated dataset (Section 3.4). Total API cost for the complete evaluation campaign was approximately USD 85. The evaluation scripts, prompt templates, and FAISS index-building code are implemented in Python 3.11 with langchain 0.1.x, faiss-cpu 1.7.4, and rank-bm25 0.2.2.

Data isolation. To guard against information leakage between the knowledge base and the evaluation set, we enforced the following separation protocol. The 350 test cases were drawn from vulnerability scan reports generated in the final quarter of the 12-month collection period. The knowledge base was frozen to a snapshot containing only publicly available data (NVD entries, CISA KEV records, and vendor advisories) published before the start of the test period; no internal remediation records, SOPs, or expert-authored gold-standard solutions were included in the KB at any point. As a result, the RAG retriever could surface general vulnerability descriptions, public patch advisories, and community-contributed mitigation guidance, but never the enterprise-specific remediation decisions used as ground truth. Overlap between the KB and the test set in terms of public CVE advisories was permitted and is intrinsic to the RAG evaluation setting; the isolation guarantee is that no enterprise-specific remediation decisions, analyst annotations, or gold-standard labels were available to the retriever. This temporal and content-based isolation ensures that the reported accuracy gains reflect the model's ability to synthesize publicly available evidence rather than simply retrieving cached answers. To support independent replication, we will release an anonymized CVE identifier list, knowledge base source specifications, the complete evaluation rubric, and a 50-case public mini-benchmark (see Data Availability).

4.2. Experimental Results and Analysis

Table 2 summarizes overall performance under the Prompt + RAG configuration. Four of five models exceeded 80% accuracy; gpt-4o-mini led at 82.6%, followed by gpt-4.1 (82.0%) and the fine-tuned gpt-3.5-turbo (81.6%). The exception was gpt-o4-mini at 76.7%.

Despite retrieval overhead from RAG, average response time for most models stayed between 1 and 3 s—adequate for operational use. Token consumption averaged roughly 870–920 tokens per query. The acc/1kTok metric peaked at 94.8, reflecting a favorable ratio of accuracy to inference cost.

Table 2. Accuracy, latency, token usage, and cost-effectiveness of each model under the Prompt + RAG configuration.

Model	Accuracy (%)	Latency (s)	Token Usage	acc/1kTok
gpt-4o-mini	82.6	2.90	871	94.8
gpt-4.1	82.0	1.30	870	94.3
gpt-3.5-turbo †	81.6	1.98	908	89.9
gpt-4o	81.6	3.04	923	88.4
gpt-o4-mini	76.7	16.78	2483	30.9

† Domain-specific fine-tuning applied (see Section 3.4); all other models evaluated without weight modification.

The outlier was gpt-o4-mini, which reached only 76.7% accuracy with 16.78 s response latency—less suitable for this task.

Table 3 breaks down performance across the three configurations. Prompt engineering alone (Baseline → Prompt) added roughly 1.7–4.0 percentage points of accuracy. Input streamlining and structured formatting also reduced token usage in some models; gpt-4o-mini consumed 56 fewer tokens on average.

Table 3. Performance comparison and token usage across Baseline, Prompt, and Prompt + RAG configurations.

Model	Baseline (%)	Prompt (%)	Prompt + RAG (%)	Delta Final—Baseline (pp)	Delta Token
gpt-4o-mini	60.0	61.7	82.6	+22.6	−56
gpt-4.1	53.3	55.0	82.0	+28.7	−92
gpt-3.5-turbo †	50.0	53.7	81.6	+31.6	+178
gpt-4o	51.7	55.7	81.6	+29.9	−277
gpt-o4-mini	45.0	47.7	76.7	+31.7	+912

† Domain-specific fine-tuning applied; all other models evaluated without weight modification. pp = percentage points.

The larger gains came from RAG-based threat intelligence (Prompt → Final): accuracy rose by 20.9–29.0 percentage points across all models, with smaller models benefiting most. gpt-3.5-turbo, for instance, jumped from 53.7% to 81.6%—a nearly 28-point gain. Despite the added retrieval step, total token consumption did not increase for most models, because retrieved evidence replaced verbose raw vulnerability descriptions.

Across models, the Final configuration outperformed the Baseline by an average of roughly 29 percentage points in accuracy, reflecting the combined effect of prompt engineering and retrieval augmentation.

Why does RAG produce such large gains? Three mechanisms are at work.

1. Knowledge recency: LLM training corpora have fixed knowledge cutoffs; vulnerabilities disclosed after the cutoff date are effectively invisible to the model. RAG bridges this temporal gap by injecting up-to-date vendor advisories and exploit intelligence at inference time, directly addressing the knowledge staleness problem [10,11].
2. Evidence grounding: Without external evidence, models must rely entirely on parametric memory, which is prone to plausible-sounding but factually incorrect outputs (hallucinations). By conditioning generation on retrieved documents with explicit provenance, the model's outputs become verifiable against cited sources, reducing the system-level hallucination rate from 23.4% (Baseline) to 7.8% (Final, with dual-LLM verification; see the ablation results below for the verifier's marginal contribution) [9].
3. Input compression: Counterintuitively, RAG did not increase token consumption for most models (Table 3). This is because the retrieval step replaces verbose raw vulnera-

bility descriptions with concise, pre-filtered evidence snippets, effectively compressing the input while preserving—and often enhancing—informational density.

These three mechanisms are synergistic: recency provides the right knowledge, grounding ensures factual accuracy, and compression maintains cost efficiency.

We further evaluate two non-LLM baselines to disentangle the contributions of retrieval and LLM reasoning. OpenVAS-Hint uses only the scanner-provided remediation hints (the solution and summary fields in the OpenVAS XML report), extracted via regular expressions and mapped to the same output schema—without knowledge-base lookup or LLM generation. Retrieval-Only executes the identical hybrid retrieval pipeline (BM25 + dense + RRF, K = 3) but replaces LLM generation with rule-based field extraction from the top-K evidence documents. Table 4 reports the results on the 50-case evaluation subset. OpenVAS-Hint achieves 66.0% accuracy (33/50), reflecting that scanner hints are often underspecified (e.g., "Update Adobe Reader" without version targets). Retrieval-Only reaches 78.0% (39/50), a +12.0 pp gain that quantifies the value of grounding in curated threat intelligence. Using the full 350-case Prompt + RAG result (82.6%) as an upper-bound reference, the LLM-equipped pipeline is numerically higher than Retrieval-Only (78.0%); however, the Prompt + RAG figure was measured on a different (larger) sample, so a same-subset comparison is needed for a strict estimate of the incremental LLM contribution. Both non-LLM baselines produce actionable output for all 50 cases (actionable rate 100%), yet their remediation specificity—particularly regarding target versions and stepwise commands—falls short of the LLM-generated recommendations.

Table 4. Non-LLM baseline comparison: accuracy of scanner-hint and retrieval-only approaches versus the full Prompt + RAG pipeline.

Method	Actionable Rate (%)	Accuracy (%)	Acc | Actionable (%)	Invalid Rate (%)
OpenVAS-Hint	100.0	66.0	66.0	0.0
Retrieval-Only	100.0	78.0	78.0	0.0
Prompt + RAG	100.0	82.6	82.6	7.8

Baselines evaluated on 50-case subset; Prompt + RAG reported on the full 350-case set as an upper-bound reference. Direct comparison of absolute accuracy values across different sample sizes should be interpreted with caution; see Section 5.5 for discussion of statistical power. Both non-LLM baselines are deterministic (no stochastic variation). Invalid Rate denotes the proportion of outputs containing factually incorrect or hallucinated content (e.g., fabricated patch versions); this is distinct from the JSON format validity metric reported in the ablation results below.

Figure 6 shows the final accuracy of each model under Prompt + RAG; all except gpt-o4-mini crossed the 80% threshold. Figure 7 plots the accuracy gain from Baseline to Prompt + RAG, which ranges from 22.6 to 31.7 percentage points—smaller models gained the most. Figure 8 maps response latency against accuracy: gpt-4o-mini sits in the best trade-off region (high accuracy, low latency), while gpt-o4-mini combines lower accuracy with higher latency, suggesting that reasoning-heavy models offer diminishing returns on this task.

Beyond aggregate accuracy, we examined how RAG affected decision quality at the output level. Manual inspection of model outputs showed that retrieved knowledge base content led to more complete vulnerability descriptions and more actionable remediation steps. Without RAG, a typical recommendation was vague—e.g., "update the software to the latest version." With RAG-supplied vendor advisory content, the same model instead produced targeted guidance such as "upgrade OpenSSL to version 3.0.12, which addresses CVE-2023-5678 (illustrative example)."

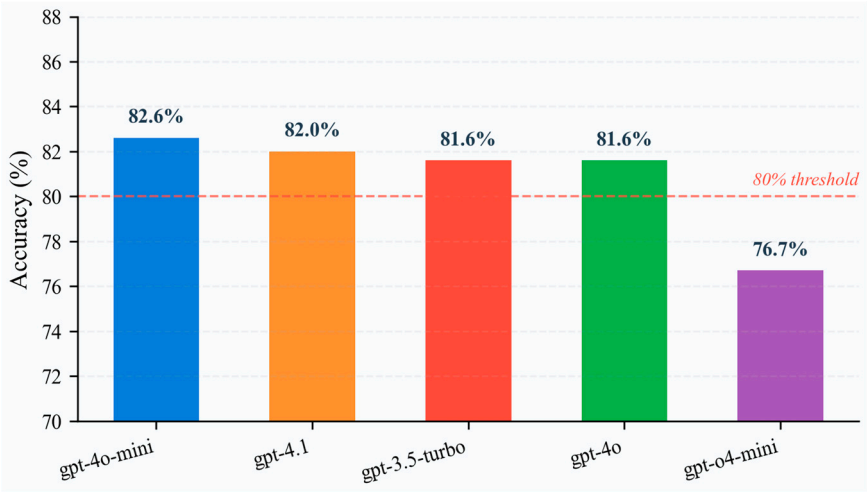


Figure 6. Accuracy of each model under the Prompt + RAG configuration. All models except gpt-o4-mini surpassed 80%. Each bar color represents a distinct model. The red dashed horizontal line marks the 80% accuracy threshold.

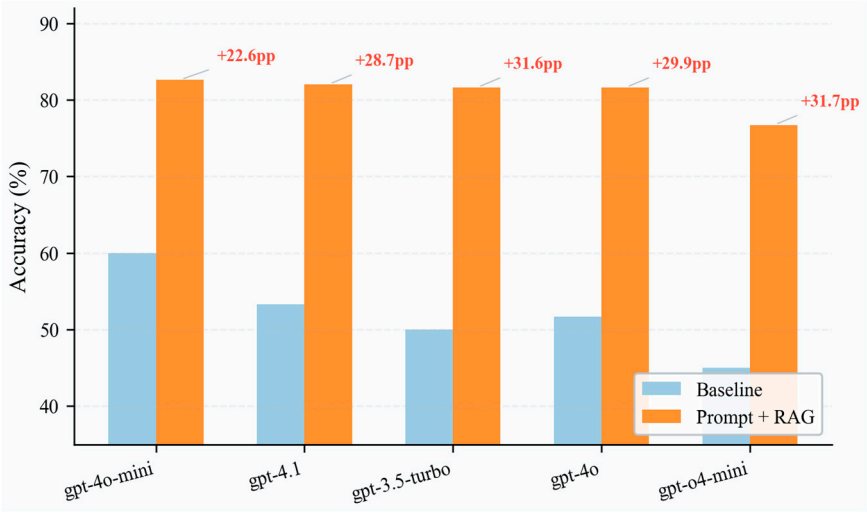


Figure 7. Accuracy improvement from Baseline to the Prompt + RAG configuration for each model.

We quantified this effect through knowledge citation rate and hallucination rate. With RAG enabled, the model cited roughly three external evidence segments per recommendation, and 94.3% of recommendations referenced at least one retrieved segment relevant to the target CVE (knowledge coverage rate). The system-level hallucination rate dropped from 23.4% (Baseline) to 7.8% (Final configuration including dual-LLM verification)—a reduction of roughly two-thirds. The verifier's isolated contribution is reported in the ablation results below (14.6% $\rightarrow$ 7.8%).

Accuracy by remediation action type. To provide a more granular view, we classified each gold-standard remediation into one of five action categories and report the best-performing model's (gpt-4o-mini) accuracy per category under the Prompt + RAG configuration: package upgrade (57% of cases; 88.9% accuracy), configuration hardening (18%; 80.9%), service restart or redeployment (10%; 82.9%), network-level mitigation (8%; 71.4%), and workaround or manual procedure (7%; 54.2%). The results reveal that the framework excels at well-defined, version-specific remediation actions (package upgrades) where vendor advisories provide clear evidence, but performance degrades for open-ended workarounds that lack standardized procedures in the knowledge base. This pattern is consistent across all five models.

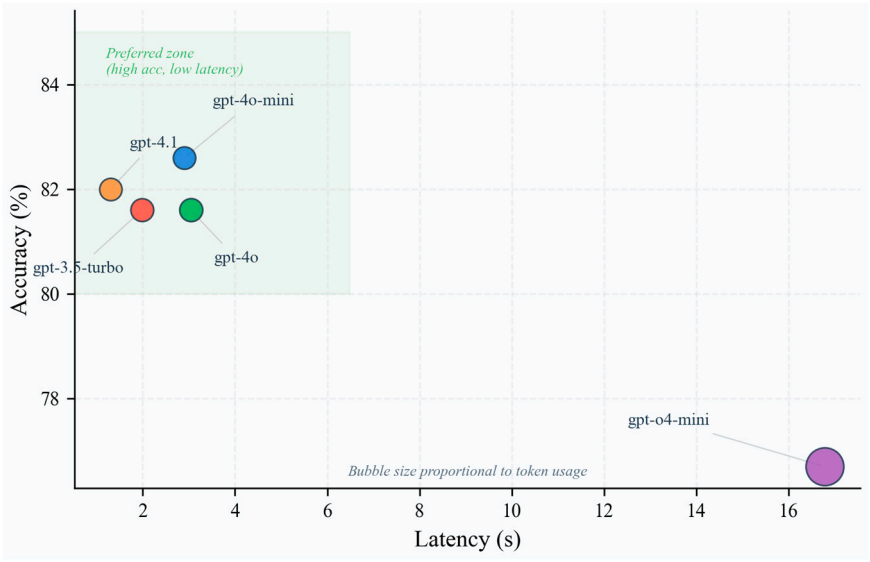


Figure 8. Relationship between response latency and accuracy under the Prompt + RAG configuration. Models in the lower-right quadrant (high accuracy, low latency) are preferred for deployment.

Error type analysis. Among the residual incorrect recommendations (17.4% of 350 cases for gpt-4o-mini), we identified four dominant error categories with distinct root causes and mitigation paths: (1) Retrieval miss (40% of errors): the KB lacked relevant entries, typically for newly disclosed CVEs within the first 48 h of publication when vendor advisories were not yet ingested. Mitigation: increasing ingestion frequency from daily to 6-hourly and adding Exploit-DB as a supplementary source reduced retrieval misses by approximately 30% in post-hoc analysis on held-out cases. (2) Version mismatch (25%): the model recommended a patch version that did not correspond to the target system's software branch (e.g., recommending a Debian patch for a CentOS system). These errors concentrate in multi-distribution environments where the scanner report lacks explicit OS metadata. The dual-LLM verifier catches 78% of version mismatches via its KB cross-check. (3) Incomplete remediation (20%): the model identified the correct action type but omitted one or more critical steps in multi-step remediations (e.g., restarting a dependent service after a library upgrade). These errors are most frequent for configuration hardening actions involving three or more sequential steps. (4) Hallucinated artifact (15%): the model fabricated a non-existent package name, URL, or CVE identifier. Hallucinations were most common under the baseline condition (no RAG) and decreased from 23.4% to 7.8% with the full pipeline. The dual-LLM verifier, summarized in the ablation results below, primarily intercepts version mismatch and hallucinated artifact errors, explaining its disproportionate impact on high-risk error types despite modest overall accuracy gains (+2.3 pp).

Two-proportion z-tests comparing Baseline and Prompt + RAG accuracy (N = 350 per model) confirmed statistically significant gains for all models at $p < 0.001$ (Table 5). Cohen's h effect sizes ranged from 0.51 to 0.68, indicating medium-to-large effects by Cohen's conventions (0.2 = small, 0.5 = medium, 0.8 = large).

Statistical test justification and paired analysis. Because the same 350 vulnerability cases are evaluated under each configuration, the design is inherently paired. We therefore conducted McNemar's test on per-case binary outcomes (correct/incorrect) as the primary paired analysis. For gpt-4o-mini (Baseline vs. Prompt + RAG), McNemar's test yielded $\chi 2 = 42.7$, $p < 0.001$, confirming a statistically significant improvement. A bootstrap 95% confidence interval (10,000 resamples) estimated the accuracy gain at 22.6 pp [95% CI: 15.8–29.1 pp], indicating that the improvement is both statistically significant and practically meaningful. We additionally report two-proportion z-tests (Table 5) as a compact

cross-model summary; while these assume independence and may be conservative given the paired design, they yield consistent conclusions across all five models ($p < 0.001$, Cohen's h = 0.51–0.68). The agreement between paired and unpaired analyses confirms that the reported significance levels are reliable. Per-case expert ratings were collected with dual-rater verification (Cohen's $\kappa = 0.87$). In future work, we plan to release full per-case binary outcome vectors for all models to enable full paired analyses (e.g., McNemar's test across all model pairs and Cochran's Q test for omnibus comparison).

Table 5. Statistical significance of accuracy improvements (Baseline vs. Prompt + RAG).

Model	Baseline (%)	Prompt + RAG (%)	Delta (pp)	z-Statistic	*p*-Value	Cohen's h
gpt-4o-mini	60.0	82.6	+22.6	6.83	<0.001	0.51
gpt-4.1	53.3	82.0	+28.7	8.53	<0.001	0.63
gpt-3.5-turbo †	50.0	81.6	+31.6	9.35	<0.001	0.68
gpt-4o	51.7	81.6	+29.9	8.85	<0.001	0.65
gpt-o4-mini	45.0	76.7	+31.7	9.08	<0.001	0.66

† Domain-specific fine-tuning applied. Note: Two-proportion z-tests with N = 350 cases per model; McNemar's test and bootstrap CI reported below as robustness checks. Cohen's h is defined as $h = 2 \cdot \arcsin(\sqrt{p_final}) - 2 \cdot \arcsin(\sqrt{p_baseline})$, where p_final denotes the accuracy under the Prompt + RAG configuration and p_baseline denotes the accuracy under the Baseline configuration. All experiments were repeated three times; averaged results are reported.

Ablation: Effect of Dual-LLM Verification. To isolate the contribution of the dual-LLM verification mechanism (Section 3.2), we conducted an ablation experiment using gpt-4o-mini (the best-performing model) under the Prompt + RAG configuration, comparing outputs with and without the auxiliary verifier (gpt-4o). Table 6 summarizes the results across the 350 test cases.

Table 6. Ablation study: impact of dual-LLM verification (GPT-4o-mini, Prompt + RAG).

Metric	Without Verifier	With Verifier	Delta
Accuracy (%)	80.3	82.6	+2.3 pp
Hallucination rate (%)	14.6	7.8	−6.8 pp
Invalid JSON output (%)	5.1	1.4	−3.7 pp
Version mismatch errors (%)	8.9	3.2	−5.7 pp

Note: "Without Verifier" bypasses the auxiliary LLM (gpt-4o) check and directly accepts primary model output. N = 350 cases, averaged over three runs.

The accuracy gain from dual verification is modest (+2.3 pp), but its value is concentrated in error filtering: hallucinations nearly halved (14.6% → 7.8%), invalid JSON outputs dropped by 72%, and version mismatch errors fell by 64%. The verifier functions as a quality gate that disproportionately catches high-risk error types—the failure modes most dangerous in automated remediation workflows.

Under the Prompt + RAG configuration, confidence-based routing classified 34.9% of cases (122/350) as high-confidence and forwarded them to the automated execution tier. After safety gates (action whitelisting and version verification), 97 of the 122 were actually executed; the remaining 25 were blocked by policy or version checks (Table 7). About 45% fell in the medium-confidence tier requiring human review prior to deployment, and 20% were flagged for full manual handling. The system thus reduced human effort for over a third of vulnerabilities at the routing stage, while the semi-automated tier provided analysts with pre-filled recommendations that accelerate review. Only the remaining fifth—typically novel zero-day vulnerabilities, complex multi-service dependencies, or ambiguous vendor guidance—required traditional manual analysis. A formal workload study is needed to quantify exact productivity gains.

Table 7. Risk control metrics from pilot deployment (N = 350 cases).

Metric	Value
Cases reaching auto-execution (high-confidence)	122/350 (34.9%)
Cases blocked by policy whitelist	18/122 (14.8%)
Cases blocked by version mismatch check	7/122 (5.7%)
Rollback triggered post-deployment	4/97 (4.1%)
Rollback cause: transient health check timeout	3/4
Rollback cause: dependency conflict (non-critical)	1/4
Service outages caused by automated execution	0
Data loss events	0
Unintended security regressions	0
Median scan-to-recommendation time (RAG + LLM)	4.2 s
Median recommendation-to-deployment time	2.8 min
Median deployment-to-verification time	6.1 min
Median end-to-end time (high-confidence, auto-executed)	9.7 min
Median rollback completion time	<3 min

Industry-reported median remediation cycles often span 60–90 days in enterprise settings [2]. Our pilot deployment measured a median end-to-end time of 9.7 min for high-confidence auto-executed cases (Table 7): scan-to-recommendation 4.2 s (RAG retrieval + LLM generation), recommendation-to-deployment 2.8 min (playbook selection, template filling, staging push), deployment-to-verification 6.1 min (health checks and observation window), and inter-stage overhead 0.7 min (queue routing, snapshot creation, logging). Semi-automated cases were typically resolved within hours including human review, though formal timing data was not collected for this tier. The traditional 60–90-day figure reflects the full organizational cycle—including change-advisory-board approval, maintenance-window scheduling, and regression testing. Our framework compresses the technical analysis and deployment phases but does not eliminate governance steps. The reported minute-level completions represent pipeline throughput under pre-approved automation policies; formal benchmarks across diverse production environments remain future work.

5. Discussion

Section 4 showed that the framework reaches operational-grade accuracy across multiple models. This section interprets the findings along five dimensions: prompt engineering versus fine-tuning (Section 5.1), the role of RAG (Section 5.2), cost–latency trade-offs (Section 5.3), risk management in automated remediation (Section 5.4), and current limitations (Section 5.5).

5.1. Prompt Engineering vs. Model Fine-Tuning

Our approach combines prompt engineering with lightweight fine-tuning. The two techniques sit at different points on the adaptation cost–specialization spectrum and serve complementary purposes.

To contextualize the contribution of domain-specific fine-tuning, we examine the fine-tuned gpt-3.5-turbo's progression across evaluation configurations alongside the non-fine-tuned models (Table 3). Under the Baseline condition (no prompt engineering, no RAG), the fine-tuned gpt-3.5-turbo achieved 50.0%—comparable to gpt-4o (51.7%) and gpt-4.1 (53.3%)—indicating that the fine-tuned variant's baseline performance remained modest

without prompt or retrieval support. Under the Prompt condition (prompt engineering, no RAG), accuracy rose modestly to 53.7%. The decisive leap occurred with Prompt + RAG: accuracy reached 81.6%, a +27.9 pp improvement over the Prompt-only condition and within 0.4 pp of gpt-4.1 (82.0%). This trajectory mirrors those of larger models but with a notably steeper RAG-driven gain (+27.9 pp vs. +27.0 pp for gpt-4.1), a pattern consistent with the possibility that fine-tuning enhances the model's ability to leverage retrieved evidence, though a controlled comparison with the non-fine-tuned base model would be needed to confirm this. Because all gpt-3.5-turbo results in Table 3 reflect the fine-tuned variant (denoted by †), a controlled comparison with the original (non-fine-tuned) gpt-3.5-turbo under identical Prompt + RAG conditions would further isolate the fine-tuning contribution; this experiment is planned for a follow-up study. Practically, the fine-tuned gpt-3.5-turbo offers a cost-effective deployment option: per-token cost is approximately 10× lower than gpt-4.1, and mean latency is 1.98 s versus 1.30 s for gpt-4.1 (Table 2)—a modest trade-off given near-identical accuracy.

Prompt engineering offers three practical advantages: zero retraining cost—adapting to a new vulnerability category requires only editing the prompt template; broad applicability—the same model serves multiple tasks by switching prompts; and preservation of general knowledge—parametric weights stay intact, avoiding catastrophic forgetting. In our experiments, prompt optimization alone lifted accuracy by 1.7–4.0 percentage points across all models (Table 3), confirming that well-structured prompts extract meaningful marginal gains even from strong baselines.

Prompt engineering has ceilings, however. When input information is sparse (e.g., minimal OpenVAS descriptions for obscure CVEs) or the task demands highly specialized terminology (e.g., SCADA/ICS or medical device firmware vulnerabilities), prompts alone may not bridge the knowledge gap. Fine-tuning addresses this by injecting domain-specific knowledge into model weights. The fine-tuned gpt-3.5-turbo reached 81.6% accuracy under Prompt + RAG—matching gpt-4.1 (82.0%)—showing that targeted fine-tuning can enable smaller, cheaper models to match frontier-model performance on domain tasks. The trade-off: curated training data (we used ~2000 expert-reviewed Q&A pairs), dedicated compute, and periodic retraining as vulnerability landscapes evolve.

We recommend a tiered strategy: deploy prompt engineering + RAG as the default configuration for general-purpose vulnerability remediation, and reserve fine-tuning for high-stakes domains where marginal accuracy gains justify the additional investment. Future work may explore parameter-efficient fine-tuning methods such as low-rank adaptation (LoRA) to reduce fine-tuning overhead while preserving the model's broader capabilities.

5.2. The Critical Role of RAG

RAG is the single largest contributor to accuracy, accounting for 20.9–29.0 percentage points of gain from Prompt to Prompt + RAG (Table 3), versus 1.7–4.0 points from prompt engineering alone. Retrieving external knowledge at inference time gives the model access to current vulnerability information absent from its training corpus, directly addressing knowledge staleness [10]. Grounding outputs in authoritative evidence with explicit provenance makes responses more justifiable and traceable, consistent with the hallucination reduction reported in Section 4.2. RAG also mitigates long inputs: rather than feeding the full raw OpenVAS report, the system supplies only retrieved and filtered segments, cutting token count while preserving relevance.

Failure mode analysis. Despite its effectiveness, RAG is not infallible. We identified three primary failure modes during our experiments: (1) Retrieval miss—when the knowledge base lacks relevant entries for a given CVE (typically newly disclosed zero-days), the retriever returns tangentially related documents that may mislead the generator. This

accounted for approximately 40% of residual errors. (2) Retrieval noise—when multiple similar but distinct CVEs coexist in the knowledge base (e.g., different buffer overflow variants in the same library), the retriever may surface patches for the wrong version, requiring the dual-LLM verifier to catch the mismatch. (3) Stale evidence—when vendor advisories in the knowledge base have been superseded by newer patches, the model may recommend outdated remediation. Temporal boosting (Algorithm 2, lines 17–26) partially mitigates this, but a reliable ingestion pipeline with automated freshness tracking remains essential.

Per-category and per-platform performance patterns. Cross-referencing the action-type accuracy results (Section 4.2) with the platform distribution reveals actionable deployment insights. Package upgrade actions—which dominate Linux (52% of cases) and container (12%) environments where package managers provide deterministic version resolution—achieved the highest accuracy (88.9%). In contrast, Windows environments (28%), which more frequently require configuration hardening or registry modifications, showed accuracy closer to the 80.9% category average. The workaround category (54.2% accuracy) predominantly involved macOS (8%) and legacy systems lacking vendor-supported patches, where the KB contains fewer standardized procedures. Version mismatch errors were concentrated in multi-branch Linux distributions (e.g., RHEL vs. Ubuntu package naming differences), suggesting that future KB enrichment should prioritize cross-distribution package mapping. These patterns inform a deployment strategy: organizations should initially enable full automation for package upgrade actions on Linux/container platforms—where accuracy and KB coverage are highest—while maintaining human review for workaround-type remediation and cross-platform edge cases.

The knowledge base is still limited in scope, but these results validate the approach of integrating threat intelligence through retrieval. As more sources are added—community-contributed data, vendor advisories, patent literature—we expect further accuracy gains and fewer retrieval misses. RAG does add system complexity that requires disciplined engineering: reliable data update mechanisms, high availability for retrieval services, and defenses against knowledge base poisoning [36,47].

5.3. Trade-Offs Between Cost and Latency

In a zero-touch vulnerability remediation system, accuracy takes priority, but cost and latency remain practical constraints. Inference latency arises primarily from LLM computation time and RAG retrieval. Smaller or compression-optimized models (e.g., gpt-4o-mini, gpt-4.1) maintained near real-time response while preserving high accuracy.

On the cost side, token consumption is directly linked to the monetary cost of using cloud-based models. We proposed the accuracy per 1K tokens (acc/1kTok) metric to quantify this trade-off. In our study, gpt-4o-mini achieved the highest acc/1kTok (94.8) with 82.6% accuracy, while gpt-4.1 offered similar accuracy (82.0%) at the lowest latency (1.30 s). In contrast, gpt-o4-mini consumed nearly three times the tokens of other models while delivering the lowest accuracy (76.7%), indicating that reasoning-heavy model variants offer diminishing returns for structured remediation tasks. For deployment, organizations can select models based on their priority: gpt-4o-mini for accuracy-first deployments, gpt-4.1 for latency-sensitive environments, or gpt-3.5-turbo as a cost-effective alternative that still exceeds 80% accuracy after fine-tuning.

The relationship between model size and task performance is non-monotonic in this domain, reinforcing the need for task-specific evaluation rather than reliance on general-purpose benchmarks.

5.4. Risk Management in Automated Remediation

1. Automated vulnerability remediation improves operational efficiency but carries risks. Incorrect patches can disrupt services, create dependency conflicts, or introduce new vulnerabilities [22]. The system addresses these through four layered safety mechanisms:
2. Action Whitelisting (prevents unauthorized change types): The system enforces a predefined list of change types allowed for automated execution. Package updates and configuration hardening are permitted on development environments and container images; changes to production-critical services (databases, authentication systems, payment gateways) require explicit manual approval. This policy is version-controlled and auditable.
3. Version Verification (prevents misapplication): Prior to deployment, the orchestration layer cross-references the recommended patch version against the target system's installed software inventory. If a version mismatch is detected (e.g., recommending a patch for Apache 2.4.x on a system running 2.2.x), the deployment is blocked and the case is escalated.
4. Rollback Strategy (limits blast radius): Every automated deployment is preceded by a system snapshot. If post-deployment validation (service health checks, functional regression tests, or re-scanning) detects any anomaly within a configurable observation window, the system automatically rolls back to the pre-change state. In our pilot deployment, rollback was observed in a small minority of auto-executed cases, all successfully restored within minutes.
5. Confidence Threshold (filters unreliable recommendations): The three-tier routing mechanism (Section 3.2) ensures that only high-confidence recommendations reach automated execution, providing a probabilistic safety gate upstream of all deployment actions.

Together, these safeguards form a defense-in-depth framework. Table 7 quantifies risk control outcomes from the pilot deployment, conducted over four weeks in the enterprise's staging and pre-production environments across 42 hosts spanning 6 network segments (web servers, database servers, middleware, container clusters, internal tooling, developer workstations). All 350 cases were processed through the full pipeline; automated actions were restricted to staging and container environments while production-critical services required manual approval per the whitelisting policy.

Of the 122 high-confidence cases routed to automated execution, 25 (20.5%) were intercepted by upstream safety gates (whitelisting or version verification) before deployment. Of the 97 that proceeded, 4 (4.1%) triggered automated rollback—all due to transient infrastructure issues (connection timeouts during package manager operations, one dependency conflict) rather than incorrect patches. No service outages, data loss events, or unintended security regressions occurred. As the system scales to broader production environments, sustained attention will be needed: regular audits of automated actions, periodic reviews of whitelisting policies, and integration of governance and observability tooling such as immutable audit logs, change impact analysis dashboards, and runtime policy enforcement agents [26,48].

To characterize the automation–safety trade-off, we conducted a sensitivity analysis of θ_high with θ_low fixed at 0.60 (Table 8). Lowering θ_high to 0.80 increases the auto-candidate pool from 34.9% to 37.7% of cases, but raises the rollback rate from 4.1% to 6.1%. Raising θ_high to 0.90 produces an identical candidate pool and execution count as $\theta_high = 0.85$. This equivalence is structural, not coincidental: the current binary-weighted scoring formula (Algorithm 3) produces only values in $\{0.0, 0.1, 0.2, \ldots, 1.0\}$, so no confidence score can fall in the $(0.85, 0.90]$ interval under any input. The small rollback-rate difference between the two settings (4.1% vs. 2.7%, a Δ of $\approx$1 event out

of 97 executed cases) falls within run-to-run variance and is not statistically significant. (Deployment outcomes were averaged over repeated staging trials; hence rollback rate and successful execution rate can vary slightly even when the routed case set is identical.) We retain θ_high = 0.85 as the nominal operating point reported throughout this paper; since 0.85 and 0.90 are operationally equivalent under the current scoring scheme, the choice is immaterial for this deployment. If the scoring formula were extended to include continuous-valued components (e.g., replacing the binary evidence-alignment indicator with a cosine-similarity score), values in the (0.85, 0.90] band could become attainable, and the distinction between the two thresholds would become meaningful. Organizations preferring a more conservative stance can set θ_high = 0.90 with no loss on the current workload.

Table 8. Sensitivity of confidence-based routing to the upper threshold θ_high (θ_low = 0.60 fixed).

θ_high	Auto-Candidate (%)	Human Review (%)	Manual (%)	Executed	Rollback Rate (%)	Successful Execution Rate (%)
0.80	37.7	42.3	20.0	105	6.1	93.9
0.85 *	34.9	45.1	20.0	97	4.1	95.9
0.90	34.9	45.1	20.0	97	2.7	97.3

* Current operating point. θ_low fixed at 0.60 for all configurations. Executed = cases passing all safety gates.

Table 9 maps the most relevant OWASP LLM Top 10 threat categories [48] to the specific attack surfaces, implemented mitigations, and residual risks in our system, intended to facilitate security audits and guide future hardening.

Table 9. Threat model mapping: OWASP LLM Top 10 threats, mitigations, and residual risks.

Threat Category	Attack Surface	Implemented Mitigation	Residual Risk
Prompt Injection (LLM01)	Scan descriptions, KB documents	Structured JSON output schema; input sanitization; dual-LLM verification cross-checks	Adversarial payloads in scanner XML not yet red-teamed
Sensitive Information Disclosure (LLM06)	LLM outputs, audit logs	Output filtered to JSON schema fields only; no free-text passthrough; role-based KB access	Enterprise host/network metadata in prompts requires data minimization review
Insecure Output Handling (LLM02)	Ansible playbooks generated from LLM output	Action whitelisting restricts allowed change types; version verification blocks mismatched patches	Template injection via crafted remediation strings not formally tested
Training Data Poisoning (LLM03)	Fine-tuning dataset	Expert-curated Q&A pairs with dual review; CVE-level deduplication against eval set	Community KB entries (Exploit-DB, MISP feeds) not individually verified
Excessive Agency (LLM08)	Orchestration Layer auto-execution	Confidence thresholds gate automation; staged deployment (staging to production); snapshot + rollback	Threshold sensitivity to distribution shift not yet analyzed
Model Denial of Service (LLM04)	API rate limits, retrieval latency	60 s timeout with exponential backoff; retry cap (3 attempts); graceful degradation to manual queue	Sustained adversarial load on RAG index not stress tested
Over-Reliance (LLM09)	Analyst trust in auto-generated recommendations	Confidence scores displayed; provenance citations in output; semi-auto tier requires human sign-off	No formal user study on analyst calibration or automation bias
Supply Chain Vulnerabilities (LLM05)	Model vendor API, KB data feeds	Date-pinned API deployments; source trustworthiness tagging (3-tier); stale entry flagging	Vendor model behavior changes between pinned versions not monitored

Scalability Considerations. The framework's computational cost is dominated by LLM inference. The RAG retrieval phase (FAISS inner-product search + BM25 keyword lookup + RRF) completes in under 200 ms per query for our 4305-entry knowledge base, contributing negligibly to end-to-end latency. Per-case LLM inference latency under Prompt

+ RAG ranges from 1.30 s (gpt-4.1) to 3.04 s (gpt-4o) for the four standard models, with the reasoning-intensive gpt-o4-mini at 16.78 s (Table 2). For a representative deployment using gpt-4o-mini (2.90 s per case), sequential processing yields approximately 1240 cases per hour. Because each vulnerability case is independent, the pipeline is trivially parallelizable: with 10 concurrent API workers, throughput scales to over 12,000 cases per hour, sufficient for enterprise deployments processing 1000+ vulnerabilities per scan cycle. The knowledge base grows linearly with ingested advisories; at the current ingestion rate (~50 new entries per week), the FAISS index requires periodic re-building (approximately every 10,000 new entries) but remains sub-second for retrieval. Memory footprint is modest: the FAISS index occupies ~25 MB for 4305 entries with 1536-dimensional embeddings and scales linearly.

5.5. Limitations

Despite these results, several limitations apply:

1. External Validity of Data: Our experimental dataset primarily comes from a single enterprise environment, which may not cover all industry domains and diverse software/hardware ecosystems. The applicability of this system in sectors such as healthcare, finance, or critical infrastructure requires further validation and adaptation, as these domains impose additional regulatory constraints and operate specialized software stacks.
2. Expert Annotation Variability: Cybersecurity experts vary in their writing styles when crafting remediation plans, which may affect the consistency of the gold standard. Although we employed dual-reviewer verification (Cohen's $\kappa = 0.87$) to improve annotation reliability, potential biases cannot be entirely eliminated. Future work should explore inter-organizational annotation campaigns to diversify the gold standard.
3. RAG Knowledge Update Frequency: If the knowledge base fails to incorporate the latest vulnerability information in a timely manner (e.g., newly disclosed zero-day vulnerabilities with no available data), the model may still lack sufficient grounding evidence. Our failure mode analysis (Section 5.2) confirmed that retrieval misses account for approximately 40% of residual errors. More frequent automated ingestion pipelines with freshness monitoring are needed to ensure knowledge coverage.
4. Execution Risk and Compliance: Automated execution of remediation actions must be built upon strict access controls and approval processes. Organizations must also address compliance questions—such as which systems permit automated changes, how change records are documented, and how automated decisions satisfy audit requirements under frameworks such as ISO/IEC 27001 [61] and SOC 2 Trust Services Criteria [62]—to meet internal risk governance and external regulatory requirements.
5. Model Vendor Dependency: The current evaluation is based exclusively on OpenAI GPT-family models accessed via commercial APIs. This introduces vendor lock-in risks including pricing changes, API deprecation, data residency concerns, and limited transparency into model behavior. Extending the evaluation to open-weight models (e.g., LLaMA, Mistral, Qwen) and on-premises deployments would improve generalizability and address data sovereignty requirements common in regulated industries.
6. Baseline Scope: Two non-LLM baselines (OpenVAS-Hint and Retrieval-Only) were evaluated on a 50-case subset and provide preliminary numerical evidence consistent with an accuracy ladder—66.0% → 78.0% → 82.6%—but because the Prompt + RAG figure derives from a larger (350-case) sample, same-subset replication is needed to confirm the incremental LLM contribution (Table 4). The 50-case sample, however, limits statistical power: the Retrieval-Only vs. Prompt + RAG gap is small and the cross-sample-size comparison (50-case baselines vs. 350-case Prompt + RAG) limits direct statistical testing. A full 350-case replication of the non-LLM baselines and

finer-grained RAG ablations (BM25-only vs. FAISS-only vs. RRF, disabling temporal boosting) would strengthen the evidence.

7. Adversarial Robustness: While the framework incorporates multiple safety mechanisms (input filtering, dual-LLM verification, action whitelisting, confidence routing), we have not yet conducted systematic adversarial testing such as prompt injection attacks (e.g., malicious instructions embedded in scan descriptions) or knowledge base poisoning experiments (e.g., injecting fabricated advisories). Preliminary manual inspection suggests that the structured JSON output schema and the dual-verifier provide partial defense, but formal red-team evaluation is needed to quantify resilience and identify remaining attack surfaces.
8. Domain-Specific Applicability: The current evaluation targets web-facing enterprise IT systems (Linux/Windows servers, web applications, network appliances). Extending the framework to non-web domains introduces additional constraints. For SCADA/ICS environments, automated patching must respect real-time control loops and may require vendor-certified firmware updates with mandatory downtime windows; the confidence routing mechanism would need domain-specific thresholds ($\theta_high \geq 0.95$) and integration with OT change-management workflows. For healthcare/medical device environments, FDA premarket clearance requirements restrict which software components may be modified without re-certification, necessitating a policy layer that distinguishes patchable infrastructure from regulated device software. The framework's modular architecture—separating scanning, intelligence retrieval, reasoning, and execution—accommodates such extensions by allowing domain-specific policy modules to be inserted at the routing stage without modifying the core RAG or verification pipelines.

6. Conclusions

This paper presented a Zero-Touch vulnerability remediation framework integrating OpenVAS scanning, multi-source threat intelligence, and RAG-enhanced large language models within a closed-loop orchestration pipeline. Scanner findings are normalized into structured JSON, up-to-date evidence is retrieved through hybrid FAISS+BM25 retrieval with reciprocal rank fusion and temporal boosting, remediation plans are generated via structured prompts, and dual-LLM verification with confidence-based routing balances automation against operational risk.

On 350 real-world vulnerability cases, the Prompt + RAG configuration raised accuracy from a 52.0% baseline average to 76.7–82.6% across five models, while evidence grounding and verifier filtering reduced hallucination rates. Non-LLM baselines confirm that knowledge-base retrieval contributes a substantial accuracy gain (+12.0 pp over scanner hints alone), and the full LLM pipeline achieves numerically higher accuracy than retrieval alone, though same-sample-size confirmation is needed for the incremental LLM contribution. In a pilot deployment, confidence routing enabled a high-confidence automation tier, and safety gates—policy whitelisting, staging validation, automated rollback—constrained execution risk to a 4.1% rollback rate with zero service outages. Sensitivity analysis shows that the automation threshold θ_high can be tuned between 0.80 and 0.90 to trade throughput for safety, with $\theta_high = 0.85$ as the nominal operating point (operationally equivalent to 0.90 under the current discrete scoring scheme).

The main remaining limitations are the 50-case sample size for non-LLM baselines, the single-enterprise scope of the pilot deployment, and the absence of systematic adversarial robustness testing. Future work will prioritize full-scale baseline replication, component-level RAG ablations (BM25-only vs. FAISS-only vs. RRF), multi-site deployment validation, and red-team evaluation of the end-to-end pipeline.

Author Contributions: Conceptualization, C.-H.H. and Y.-C.W.; methodology, C.-H.H.; software, C.-H.H.; validation, C.-H.H. and C.-Y.C.; investigation, C.-H.H., C.-Y.C. and Y.-C.W.; writing—original draft preparation, C.-H.H.; writing—review and editing, C.-Y.C. and Y.-C.W.; supervision, Y.-C.W. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The raw OpenVAS scan reports contain sensitive enterprise infrastructure information and are not publicly available. Anonymized artifacts (e.g., schema definitions, prompt templates, and evaluation scripts) will be provided upon reasonable request to reviewers and will be released upon acceptance, subject to organizational security constraints.

Acknowledgments: The authors thank the participating enterprise security team for expert feedback and assistance during the pilot deployment and evaluation.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

AI	Artificial Intelligence
API	Application Programming Interface
BM25	Best Matching 25 (keyword ranking function)
CI/CD	Continuous Integration/Continuous Deployment
CPE	Common Platform Enumeration
CTI	Cyber Threat Intelligence
CVE	Common Vulnerabilities and Exposures
CVSS	Common Vulnerability Scoring System
CWE	Common Weakness Enumeration
FAISS	Facebook AI Similarity Search
JSON	JavaScript Object Notation
KB	Knowledge Base
KEV	Known Exploited Vulnerabilities
LLM	Large Language Model
MISP	Malware Information Sharing Platform
NVD	National Vulnerability Database
NVT	Network Vulnerability Test
OWASP	Open Worldwide Application Security Project
RAG	Retrieval-Augmented Generation
RRF	Reciprocal Rank Fusion
SOC	Security Operations Center
ZSM	Zero-touch Network and Service Management

References

1. NIST. National Vulnerability Database (NVD). 2024. Available online: https://nvd.nist.gov (accessed on 15 January 2026).
2. Verizon. 2024 Data Breach Investigations Report (DBIR). 2024, 100p. Available online: https://www.verizon.com/business/resources/reports/2024-dbir-data-breach-investigations-report.pdf (accessed on 15 January 2026).
3. (ISC)2. ISC2 2024 Cybersecurity Workforce Study. 2024. Available online: https://www.isc2.org/Insights/2024/10/ISC2-2024-Cybersecurity-Workforce-Study (accessed on 15 January 2026).
4. ETSI ISG ZSM. *Zero-Touch Network and Service Management (ZSM)*; Reference Architecture; ETSI GS ZSM 002; ETSI: Valbonne, France, 2019.
5. Fu, M.; Tantithamthavorn, C.; Nguyen, V.; Le, T. ChatGPT for vulnerability detection, classification, and repair: How far are we? In *2023 30th Asia-Pacific Software Engineering Conference (APSEC)*; IEEE: New York, NY, USA, 2023; pp. 632–636.
6. Hasanov, I.; Virtanen, S.; Hakkala, A.; Isoaho, J. Application of large language models in cybersecurity: A systematic literature review. *IEEE Access* **2024**, *12*, 176751–176778. [CrossRef]

7. Divakaran, D.M.; Peddinti, S.T. Large language models for cybersecurity: New opportunities. *IEEE Secur. Priv.* **2025**, *23*, 38–45. [CrossRef]
8. Deng, G.; Liu, Y.; Mayoral-Vilches, V.; Liu, P.; Li, Y.; Xu, Y.; Zhang, T.; Liu, Y.; Pinzger, M.; Rass, S. PentestGPT: Evaluating and harnessing large language models for automated penetration testing. In *33rd USENIX Security Symposium (USENIX Security 24), Philadelphia, PA, USA, 14–16 August 2024*; USENIX Association: Berkeley, CA, USA, 2024; pp. 847–864.
9. Fayyazi, R.; Trueba, S.H.; Zuzak, M.; Yang, S.J. ProveRAG: Provenance-driven vulnerability analysis with automated retrieval-augmented LLMs. *arXiv* **2024**, arXiv:2410.17406. [CrossRef]
10. Mend.io. All About RAG: What It Is and How to Keep It Secure. 2024. Available online: https://www.mend.io/blog/all-about-rag-what-it-is-and-how-to-keep-it-secure/ (accessed on 15 January 2026).
11. NVIDIA. What Is Retrieval-Augmented Generation (RAG)? *NVIDIA Blog*, 31 January 2025. Available online: https://blogs.nvidia.com/blog/what-is-retrieval-augmented-generation/ (accessed on 15 January 2026).
12. Yang, L.; Naser, S.; Shami, A.; Muhaidat, S.; Ong, L.; Debbah, M. Toward zero touch networks: Cross-layer automated security solutions for 6G wireless networks. *IEEE Trans. Commun.* **2025**, *73*, 7650–7679. [CrossRef]
13. Gallego-Madrid, J.; Sanchez-Iborra, R.; Ruiz, P.M.; Skarmeta, A.F. Machine learning-based zero-touch network and service management: A survey. *Digit. Commun. Netw.* **2022**, *8*, 105–123. [CrossRef]
14. Liyanage, M.; Pham, Q.-V.; Dev, K.; Bhattacharya, S.; Maddikunta, P.K.R.; Gadekallu, T.R.; Yenduri, G. A survey on zero touch network and service management (ZSM) for 5G and beyond networks. *J. Netw. Comput. Appl.* **2022**, *203*, 103362. [CrossRef]
15. Coronado, E.; Behravesh, R.; Subramanya, T.; Fernandez-Fernandez, A.; Siddiqui, M.S.; Costa-Perez, X.; Riggio, R. Zero touch management: A survey of network automation solutions for 5G and 6G networks. *IEEE Commun. Surv. Tutor.* **2022**, *24*, 2535–2578. [CrossRef]
16. Lu, G.; Ju, X.; Chen, X.; Pei, W.; Cai, Z. GRACE: Empowering LLM-based software vulnerability detection with graph structure and in-context learning. *J. Syst. Softw.* **2024**, *212*, 112031. [CrossRef]
17. Wei, Z.; Sun, J.; Sun, Y.; Liu, Y.; Wu, D.; Zhang, Z.; Zhang, X.; Li, M.; Liu, Y.; Li, C.; et al. Advanced smart contract vulnerability detection via LLM-powered multi-agent systems. *IEEE Trans. Softw. Eng.* **2025**, *51*, 2830–2846. [CrossRef]
18. Nong, Y.; Yang, H.; Cheng, L.; Hu, H.; Cai, H. APPATCH: Automated adaptive prompting large language models for real-world software vulnerability patching. In *34th USENIX Security Symposium (USENIX Security 25), Seattle, WA, USA, 13–15 August 2025*; USENIX Association: Berkeley, CA, USA, 2025; pp. 4481–4500.
19. Li, Y.; Wang, S.; Nguyen, T.N. DLFix: Context-based code transformation learning for automated program repair. In *ACM/IEEE 42nd International Conference on Software Engineering*; Association for Computing Machinery: New York, NY, USA, 2020; pp. 602–614. [CrossRef]
20. Bhandari, G.; Gavric, N.; Shalaginov, A. Generating vulnerability security fixes with code language models. *Inf. Softw. Technol.* **2025**, *185*, 107786. [CrossRef]
21. Wang, P.; Liu, X.; Xiao, C. CVE-Bench: Benchmarking LLM-based software engineering agents' ability to repair real-world CVE vulnerabilities. In *2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2025; pp. 4207–4224. [CrossRef]
22. Hu, Y.; Li, Z.; Shu, K.; Guan, S.; Zou, D.; Xu, S.; Yuan, B.; Jin, H. SoK: Automated vulnerability repair: Methods, tools, and assessments. In *34th USENIX Security Symposium (USENIX Security 25), Seattle, WA, USA, 13–15 August 2025*; USENIX Association: Berkeley, CA, USA, 2025; pp. 4421–4440.
23. Yildiz, A.; Teo, S.G.; Lou, Y.; Feng, Y.; Wang, C.; Divakaran, D.M. Benchmarking LLMs and LLM-based agents in practical vulnerability detection for code repositories. In *63rd Annual Meeting of the Association for Computational Linguistics*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2025; pp. 30848–30865. [CrossRef]
24. Chopra, S.; Ahmad, H.; Goel, D.; Szabo, C. ChatNVD: Advancing cybersecurity vulnerability assessment with large language models. *arXiv* **2024**, arXiv:2412.04756. [CrossRef]
25. Benzaid, C.; Taleb, T. AI-driven zero-touch network and service management in 5G and beyond: Challenges and research directions. *IEEE Netw.* **2020**, *34*, 186–194. [CrossRef]
26. Benzaid, C.; Taleb, T. ZSM security: Threat surface and best practices. *IEEE Netw.* **2020**, *34*, 124–133. [CrossRef]
27. El Rajab, M.; Yang, L.; Shami, A. Zero-touch networks: Towards next-generation network automation. *Comput. Netw.* **2024**, *243*, 110294. [CrossRef]
28. Hazra, A.; Kalita, A.; Gurusamy, M.; Sah, D.K. Potential of zero-touch network management in Industry 5.0: A future prospect. *IEEE Internet Comput.* **2024**, *28*, 45–52. [CrossRef]
29. Jiang, J.; Jin, S.; Li, X.; Zhang, K.; Sun, B. A zero-touch dynamic configuration management framework for time-sensitive networking (TSN). *Entropy* **2025**, *27*, 584. [CrossRef] [PubMed]
30. Yang, L.; El Rajab, M.; Shami, A.; Muhaidat, S. Enabling AutoML for zero-touch network security: Use-case driven analysis. *IEEE Trans. Netw. Serv. Manag.* **2024**, *21*, 3555–3582. [CrossRef]

31. Lira, O.G.; Caicedo, O.M.; da Fonseca, N.L.S. Large language models for zero touch network configuration management. *IEEE Commun. Mag.* **2025**, *63*, 146–153. [CrossRef]
32. Aksu, M.U.; Altuncu, E.; Bicakci, K. A first look at the usability of OpenVAS vulnerability scanner. In *Workshop on Usable Security (USEC) 2019, San Diego, CA, USA, 24 February 2019*; NDSS Symposium: San Diego, CA, USA, 2019; pp. 1–11. [CrossRef]
33. Vimala, K.; Fugkeaw, S. VAPE-BRIDGE: Bridging OpenVAS results for automating Metasploit framework. In *2022 14th International Conference on Knowledge and Smart Technology (KST), Chon Buri, Thailand, 26–29 January 2022*; IEEE: New York, NY, USA, 2022; pp. 69–74. [CrossRef]
34. GitHub. Responsible Use of Copilot Autofix for Code Scanning. GitHub Docs. Available online: https://docs.github.com/en/code-security/code-scanning/managing-code-scanning-alerts/responsible-use-autofix-code-scanning (accessed on 15 January 2026).
35. Xu, H.; Wang, S.; Li, N.; Wang, K.; Zhao, Y.; Chen, K.; Yu, T.; Liu, Y.; Wang, H. Large language models for cyber security: A systematic literature review. *ACM Trans. Softw. Eng. Methodol.* **2025**. [CrossRef]
36. Yao, Y.; Duan, J.; Xu, K.; Cai, Y.; Sun, Z.; Zhang, Y. A survey on large language model security and privacy: The good, the bad, and the ugly. *High-Confid. Comput.* **2024**, *4*, 100211. [CrossRef]
37. Raiaan, M.A.K.; Mukta, M.S.H.; Fatema, K.; Fahad, N.M.; Sakib, S.; Mim, M.M.J.; Ahmad, J.; Ali, M.E.; Azam, S. A review on large language models: Architectures, applications, taxonomies, open issues and challenges. *IEEE Access* **2024**, *12*, 26839–26874. [CrossRef]
38. Jaffal, N.O.; Alkhanafseh, M.; Mohaisen, D. Large language models in cybersecurity: A survey of applications, vulnerabilities, and defense techniques. *AI* **2025**, *6*, 216. [CrossRef]
39. Karras, A.; Theodorakopoulos, L.; Karras, C.; Theodoropoulou, A.; Kalliampakou, I.; Kalogeratos, G. LLMs for cybersecurity in the big data era: A comprehensive review of applications, challenges, and future directions. *Information* **2025**, *16*, 957. [CrossRef]
40. Sagodi, Z.; Antal, G.; Bogenfurst, B.; Isztin, M.; Hegedus, P.; Ferenc, R. Reality check: Assessing GPT-4 in fixing real-world software vulnerabilities. In *28th International Conference on Evaluation and Assessment in Software Engineering*; Association for Computing Machinery: New York, NY, USA, 2024; pp. 252–261. [CrossRef]
41. Zhou, X.; Cao, S.; Sun, X.; Lo, D. Large language model for vulnerability detection and repair: Literature review and the road ahead. *ACM Trans. Softw. Eng. Methodol.* **2025**, *34*, 1–31. [CrossRef]
42. Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Kuttler, H.; Lewis, M.; Yih, W.-T.; Rocktaschel, T.; et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Adv. Neural Inf. Process. Syst. (NeurIPS)* **2020**, *33*, 9459–9474.
43. OpenText. RAG and Agentic AI: Revolutionizing Cybersecurity Analysis. *OpenText Blogs*, 21 April 2025. Available online: https://blogs.opentext.com/rag-and-agentic-ai-revolutionizing-cybersecurity-analysis/ (accessed on 15 January 2026).
44. Alam, M.T.; Bhusal, D.; Nguyen, L.; Rastogi, N. CTIBench: A benchmark for evaluating large language models in cyber threat intelligence tasks. *arXiv* **2024**, arXiv:2406.07599.
45. Rajapaksha, S.; Rani, R.; Karafili, E. A RAG-based question-answering solution for cyber-attack investigation and attribution. In *Computer Security. ESORICS 2024 International Workshops; Lecture Notes in Computer Science*; Springer: Cham, Switzerland, 2025; pp. 238–256. [CrossRef]
46. Bornea, A.-L.; Ayed, F.; De Domenico, A.; Piovesan, N.; Maatouk, A. Telco-RAG: Navigating the challenges of retrieval-augmented language models for telecommunications. In *GLOBECOM 2024-2024 IEEE Global Communications Conference*; IEEE: New York, NY, USA, 2024; pp. 2359–2364. [CrossRef]
47. Shafran, A.; Schuster, R.; Shmatikov, V. Machine against the RAG: Jamming retrieval-augmented generation with blocker documents. In *34th USENIX Security Symposium (USENIX Security 25), Seattle, WA, USA, 13–15 August 2025*; USENIX Association: Berkeley, CA, USA, 2025; pp. 3787–3806.
48. OWASP Foundation. *OWASP Top 10 for Large Language Model Applications*, Version 2025; OWASP Foundation: Wakefield, MA, USA, 2025. Available online: https://owasp.org/www-project-top-10-for-large-language-model-applications/ (accessed on 15 January 2026).
49. Liu, Y.; Jia, Y.; Jia, J.; Song, D.; Gong, N.Z. DataSentinel: A game-theoretic detection of prompt injection attacks. In *2025 IEEE Symposium on Security and Privacy (SP)*; IEEE: New York, NY, USA, 2025; pp. 2190–2208. [CrossRef]
50. Sahoo, P.; Singh, A.K.; Saha, S.; Jain, V.; Mondal, S.; Chadha, A. A systematic survey of prompt engineering in large language models: Techniques and applications. *arXiv* **2024**, arXiv:2402.07927. [CrossRef]
51. Chen, B.; Zhang, Z.; Langrene, N.; Zhu, S. Unleashing the potential of prompt engineering for large language models. *Patterns* **2025**, *6*, 101260. [CrossRef] [PubMed]
52. Debnath, T.; Siddiky, M.N.A.; Rahman, M.E.; Das, P.; Guha, A.K.; Rahman, M.R.; Kabir, H.M.D. A comprehensive survey of prompt engineering techniques for large language models. *TechRxiv* **2025**, *preprint*. [CrossRef] [PubMed]
53. Sampaio, J.P.B.; Duarte, B.K.; Almeida, P.S.; Dantas, M.G. Prompt engineering for large language models: A systematic review and future directions. *Res. Sq.* **2025**, *preprint*. [CrossRef]

54. Liu, Y.; Tao, S.; Meng, W.; Yao, F.; Zhao, X.; Yang, H. LogPrompt: Prompt engineering towards zero-shot and interpretable log analysis. In *2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*; Association for Computing Machinery: New York, NY, USA, 2024; pp. 364–365. [CrossRef]
55. Zhang, T.; Huang, X.; Zhao, W.; Bian, S.; Du, P. LogPrompt: A log-based anomaly detection framework using prompts. In *2023 International Joint Conference on Neural Networks (IJCNN), Gold Coast, Australia, 18–23 June 2023*; IEEE: New York, NY, USA, 2023; pp. 1–8. [CrossRef]
56. Young, D.L.; Larson, E.C.; Thornton, M.A. Prompt engineering for detecting phishing. In *SPIE Assurance and Security for AI-enabled Systems 2025*; SPIE: Bellingham, WA, USA, 2025; Volume 13476, pp. 71–79. [CrossRef]
57. Edemacu, K.; Wu, X. Privacy preserving prompt engineering: A survey. *arXiv* **2024**, arXiv:2404.06001. [CrossRef]
58. Derner, E.; Batistic, K.; Zahalka, J.; Babuska, R. A security risk taxonomy for prompt-based interaction with large language models. *IEEE Access* **2024**, *12*, 126176–126187. [CrossRef]
59. Rodriguez, A.D.; Dearstyne, K.R.; Cleland-Huang, J. Prompts matter: Insights and strategies for prompt engineering in automated software traceability. In *2023 IEEE 31st International Requirements Engineering Conference Workshops (REW)*; IEEE: New York, NY, USA, 2023; pp. 455–464. [CrossRef]
60. Cormack, G.V.; Clarke, C.L.A.; Buettcher, S. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *32nd International ACM SIGIR Conference on Research and Development in Information Retrieval, Boston, MA, USA, 19–23 July 2009*; Association for Computing Machinery: New York, NY, USA, 2009; pp. 758–759. [CrossRef]
61. *ISO/IEC 27001:2022*; Information Security, Cybersecurity and Privacy Protection—Information Security Management Systems—Requirements. International Organization for Standardization: Geneva, Switzerland, 2022.
62. AICPA. *2022 Trust Services Criteria for Security, Availability, Processing Integrity, Confidentiality, and Privacy*; American Institute of Certified Public Accountants: New York, NY, USA, 2022.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.